CSCI 350:
Fundamentals of Operating Systems

Context switch, System calls

Prof. Tatyana Ryutov

This content is protected and may not be shared, uploaded, or distributed.

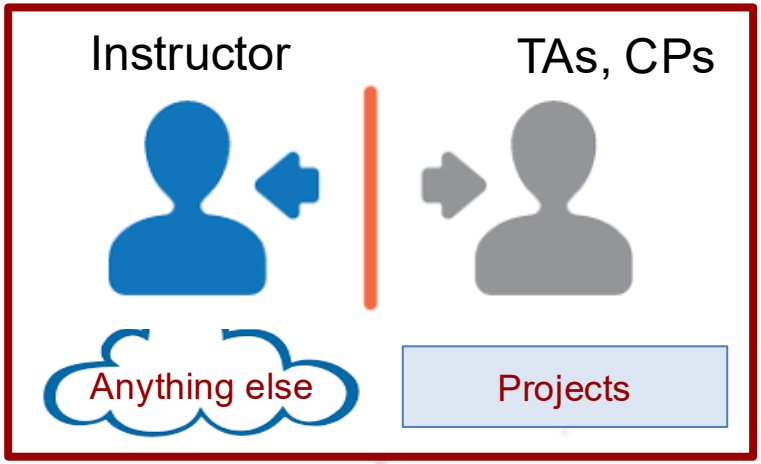
Lecture Outline

- Review
- Dual mode operation
- Event processing
- Context and process switch
- System calls
- Kernel organization

©Tatyana Ryutov

Reminders

- Division of responsibilities:
  - Programming projects – CPs and TAs
  - Exam grading – TAs
  - Anything else – instructor
    - Help with the course material
    - Any other issues (excluding programming projects)
  - Students
    - Ask and answer questions on Piazza
- The course team is available to answer your email/Piazza questions 9:00am-6:00pm on weekdays only

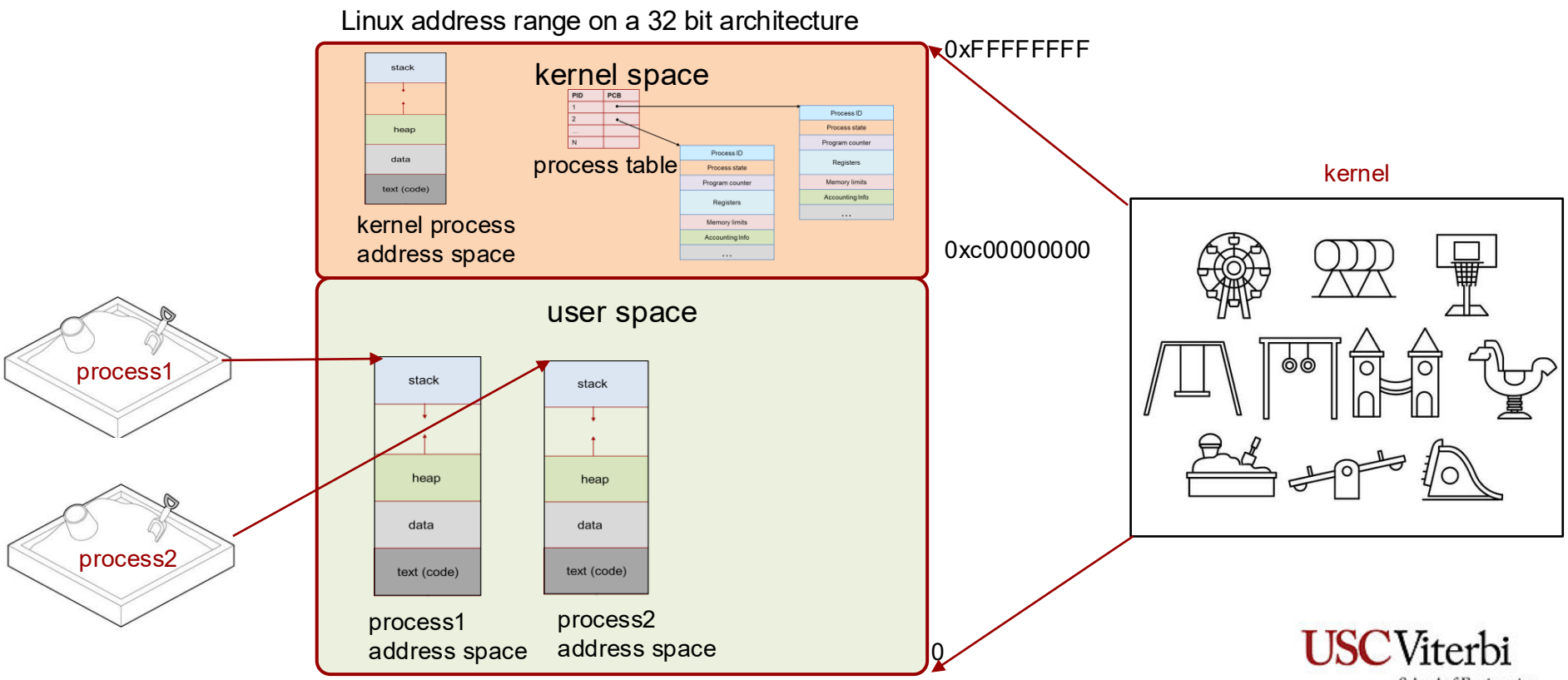


Class will proceed at a brisk pace and often be more focused on providing extra intuition and discussions rather than rehashing the textbook content in great detail
Read before class!

©Tatyana Ryutov

Review: Address Space Abstraction

- Memory is divided into two distinct regions:
  1. Kernel space is where the kernel (the core of the OS) runs and provides its services
     - Can access all memory and HW
  2. User space is where user processes run (i.e. everything other than the kernel)
     - Cannot access kernel space directly

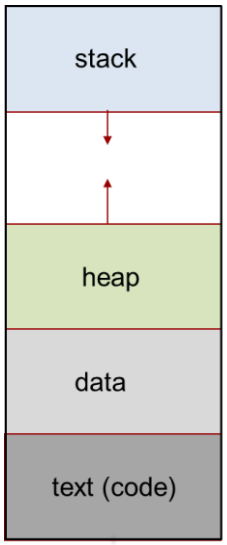


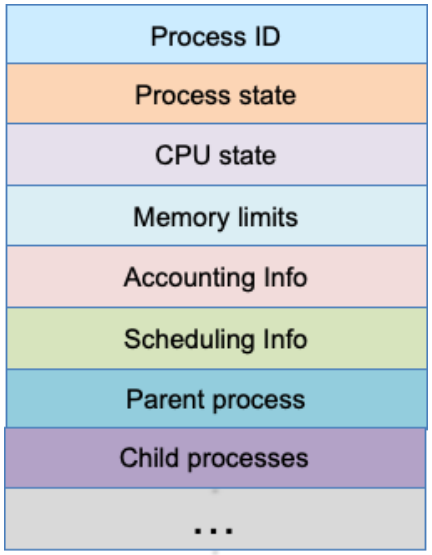
©Tatyana Ryutov

USC Viterbi School of Engineering

Review: Process Abstraction

- Process is an instance of a program, running with limited rights
- Process consists of two parts:
  1. Address space that contains:
     - Code
     - Data
     - Stack
     - Heap
     - Kernel stack
  2. Process Control Block (PCB) that contains:
     - Registers, program counter, list of open files, etc.





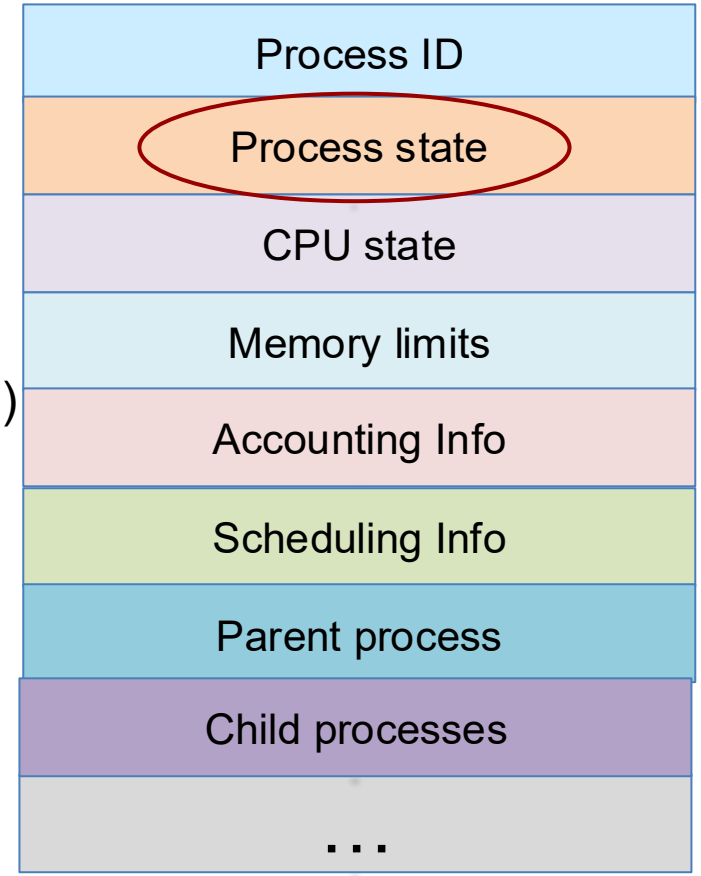
©Tatyana Ryutov

USC Viterbi School of Engineering

Recall: PCB

- OS maintains information about every process in a data structure called a Process Control Block (PCB)

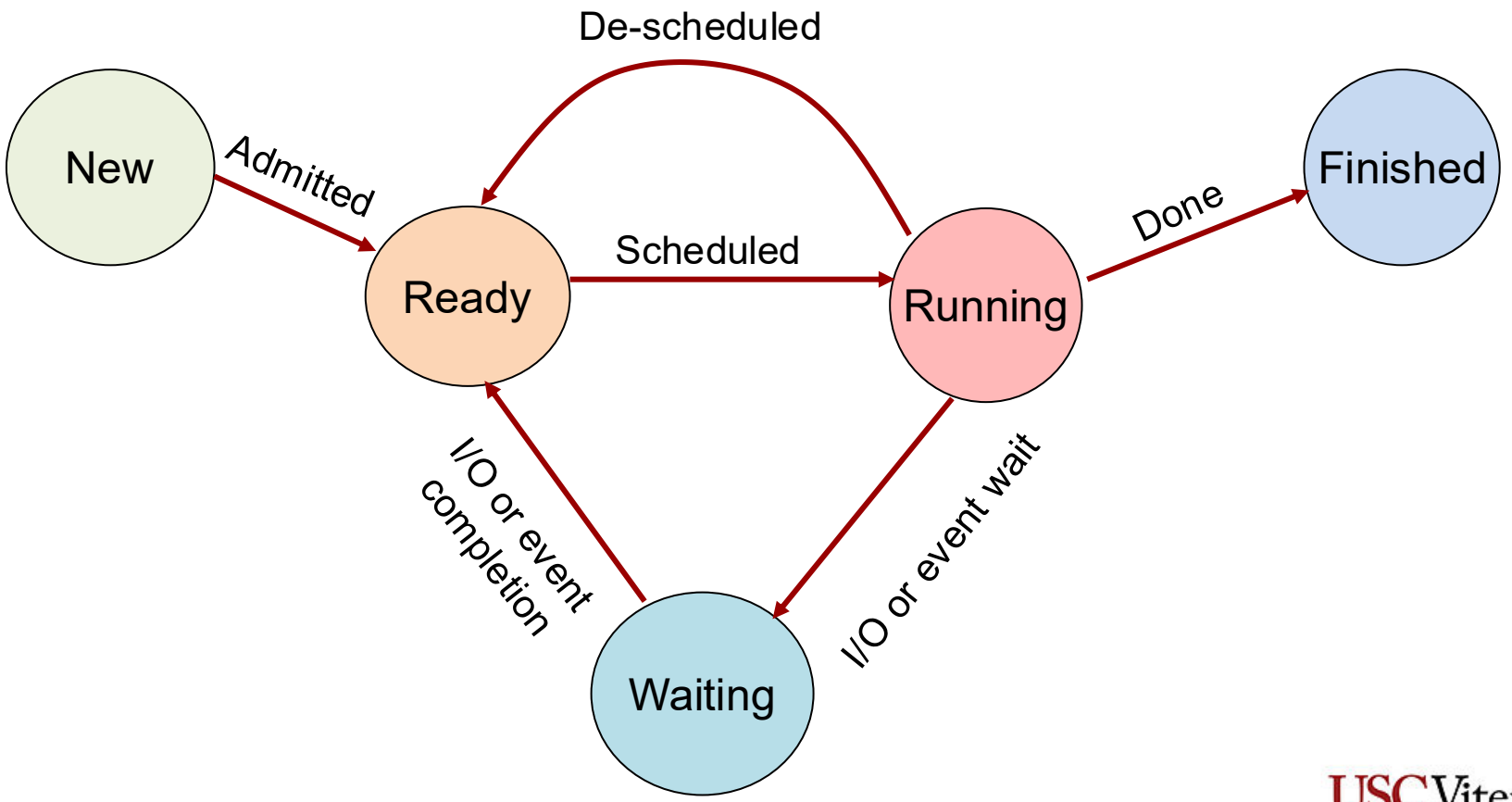
- PCB contains:
  - Unique process identifier
  - Process state (Running, Ready, etc.)
  - CPU state (program counter, registers, flags)
  - Memory management information (e.g., page tables, segment tales, base & bound registers)
  - CPU scheduling & accounting information
  - Parent process
  - Child processes
  - ...



©Tatyana Ryutov

Process States

- As a process executes, it changes its state
- A process may be in one of the following states:

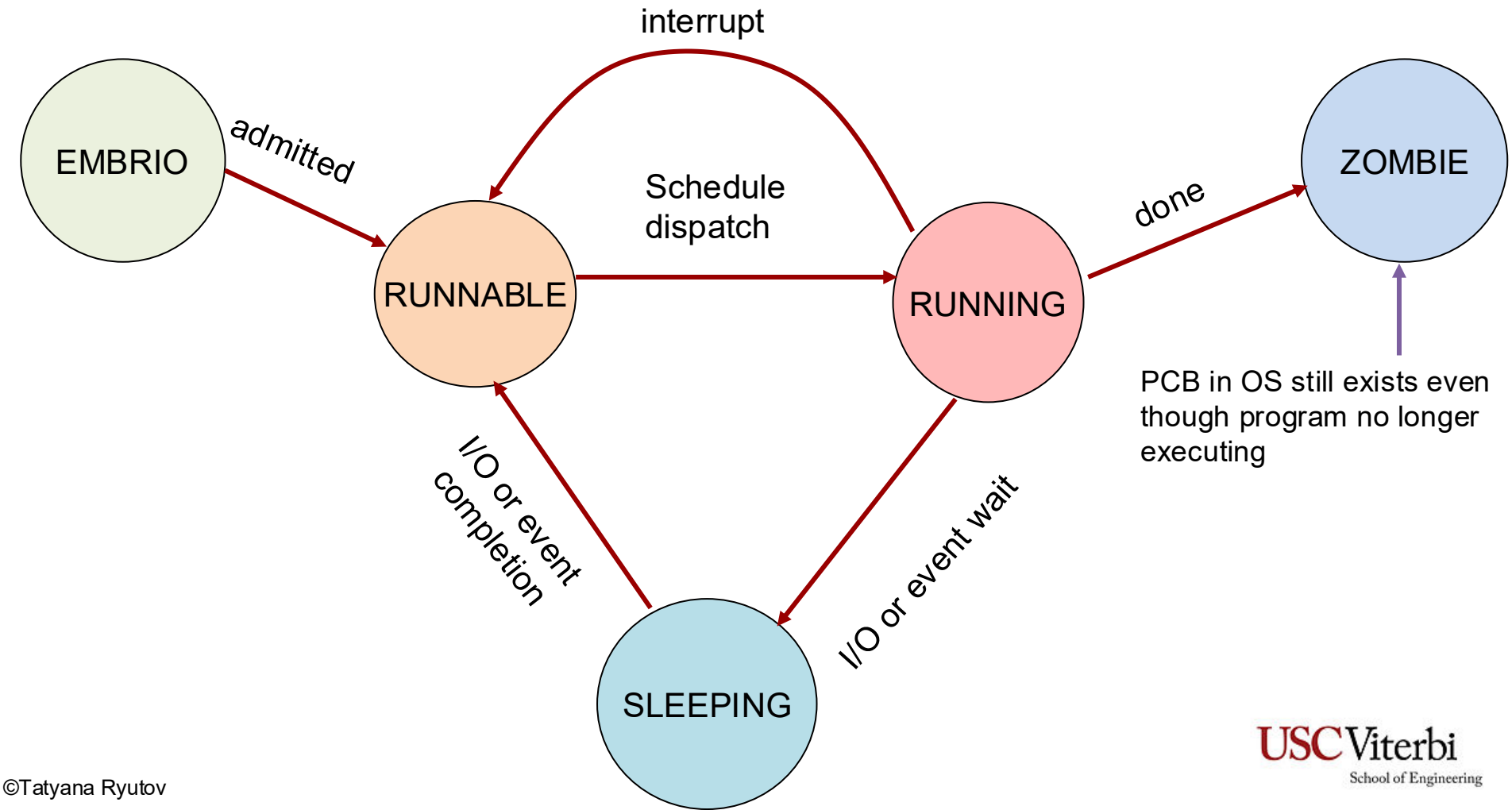


©Tatyana Ryutov

USC Viterbi School of Engineering

xv6 Process States

```
proc.h:
enum procstate {UNUSED,EMBRYO,SLEEPING,RUNNABLE,RUNNING, ZOMBIE};
```

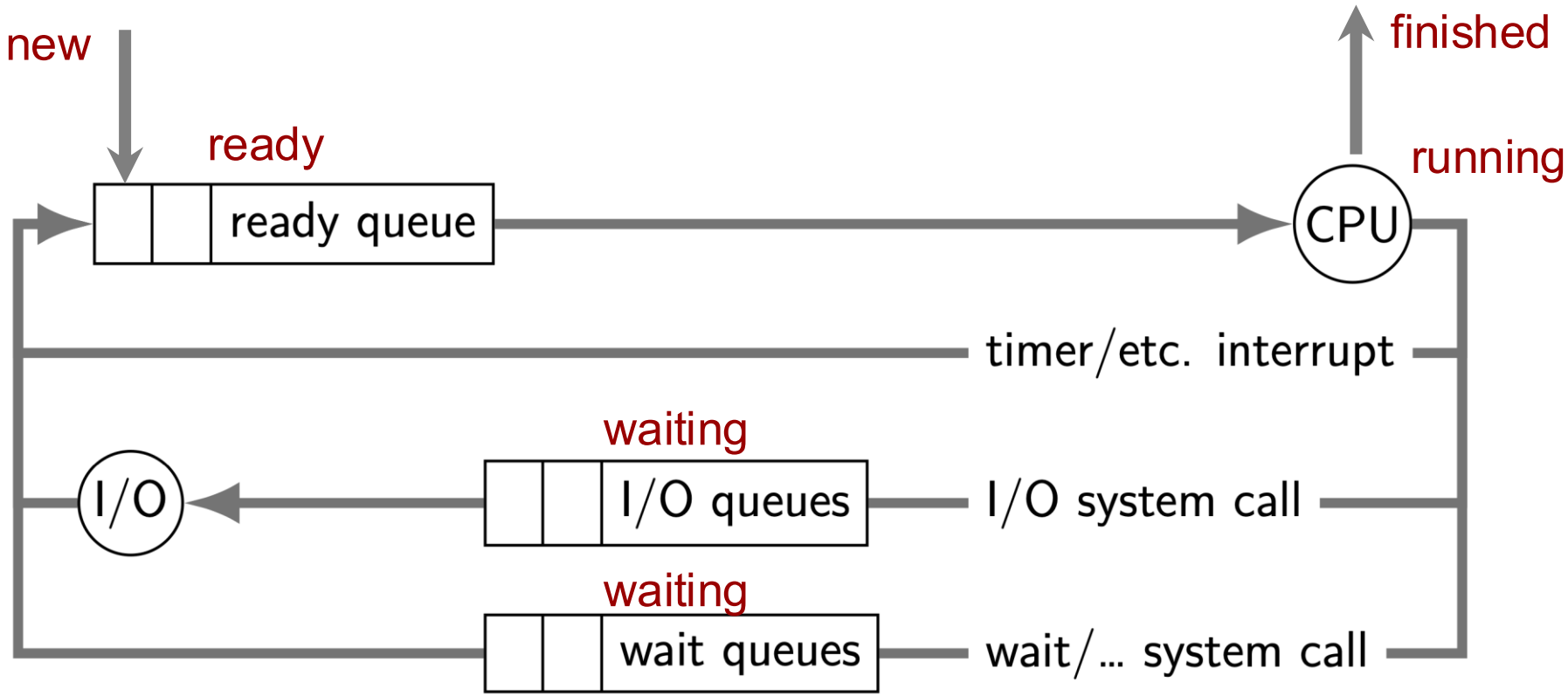


©Tatyana Ryutov

USC Viterbi School of Engineering

Alternative View: Process Queues

- OS manages various types of queues for process states
  - Process's PCB is stored in the queue of the corresponding state
  - If process is moved from one state to another, its PCB is unlinked from the corresponding queue and added to the new state queue



©Tatyana Ryutov

USC Viterbi School of Engineering

Process Queues in xv6

- xv6 does not represent queues explicitly, no queue class/struct
- Ready queue: process list ignoring non-RUNNABLE entries
- I/O queues: process list where SLEEPING, chan = I/O device
- Real OSs: typically separate list of processes

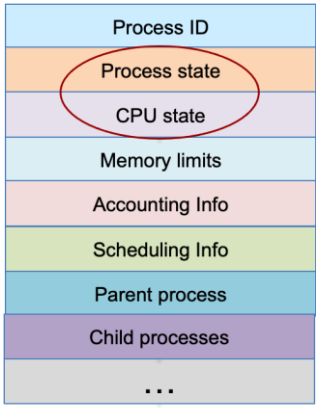
```
struct {
   struct spinlock loc;
   struct proc proc[NPROC];
} ptable;
```

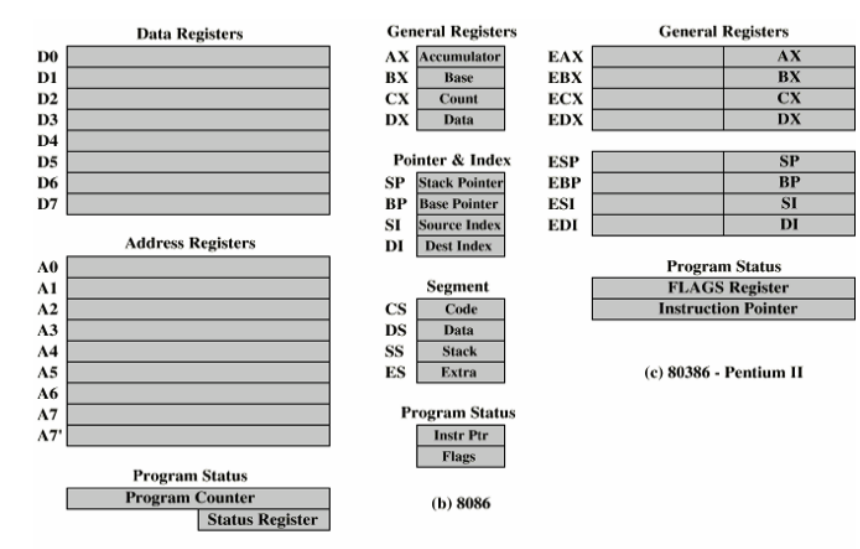
©Tatyana Ryutov

CPU State

- If you run `info registers` in gdb, the output will be similar to the following:

```
(gdb) start
Temporary breakpoint 1 at 0x80483f0: file test.cpp, line 5.
Starting program: /home/bazis/test

Temporary breakpoint 1, main (argc=1, argv=0xbffff064) at test.cpp:5
5 return 0;
(gdb) info registers
eax 0x1 1
ecx 0xbffff064 -1073745820
edx 0x80483ed 134513645
ebx 0xb7fbe000 -1208229888
esp 0xbfffefc8 0xbfffefc8
ebp 0xbfffefc8 0xbfffefc8
esi 0x0 0
edi 0x0 0
eip 0x80483f0 0x80483f0 <main(int, char**)+3>
eflags 0x246 [ PF ZF IF ]
cs 0x73 115
ss 0x7b 123
ds 0x7b 123
es 0x7b 123
fs 0x0 0
gs 0x33 51
(gdb) info registers eax
eax 0x1 1
(gdb) info registers cx
cx 0xf064 -3996
```





©Tatyana Ryutov

Lecture Outline

- Review
- Dual mode operation
- Event processing
- Context and process switch
- System calls
- Kernel organization

©Tatyana Ryutov

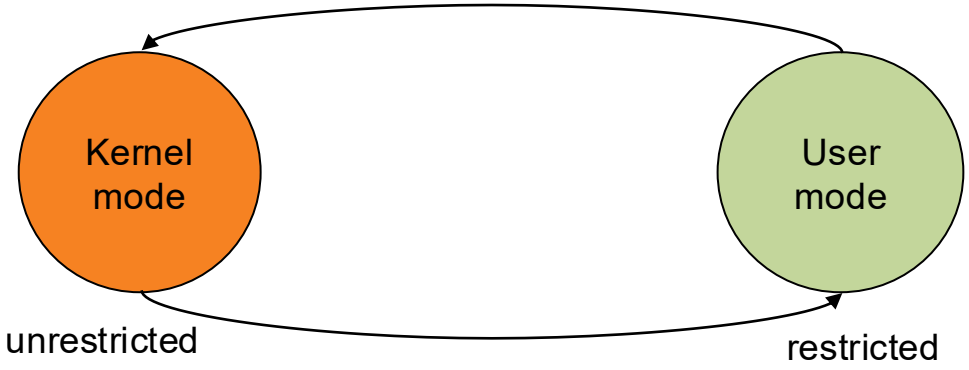
Goal: Protection

- Need for protection
  - Kernel cannot trust user processes which may be malicious or buggy
  - Kernel must protect
    - User processes from one another
    - Kernel from user processes
  - Isolating processes from one another is the key to protection
- HW mechanisms for protection:
  1. Memory protection (will cover later)
     - Segmentation and paging
  2. Timer interrupt
     - Kernel periodically gets back control
  3. Dual mode of operation
     - Privileged (+ non-privileged) operations in kernel mode
     - Non-privileged operations in user mode

©Tatyana Ryutov

Dual Mode Operation

- **Goal**: distinguish between execution of trusted OS code and untrusted user code
- Two processor modes:
  1. Kernel mode
     - Execution with the full privileges of the HW
  2. User mode
     - Limited privileges and memory accesses
- Processor operates either in user mode or kernel mode
  - Applications operate in user mode, while core OS components function in kernel mode



©Tatyana Ryutov

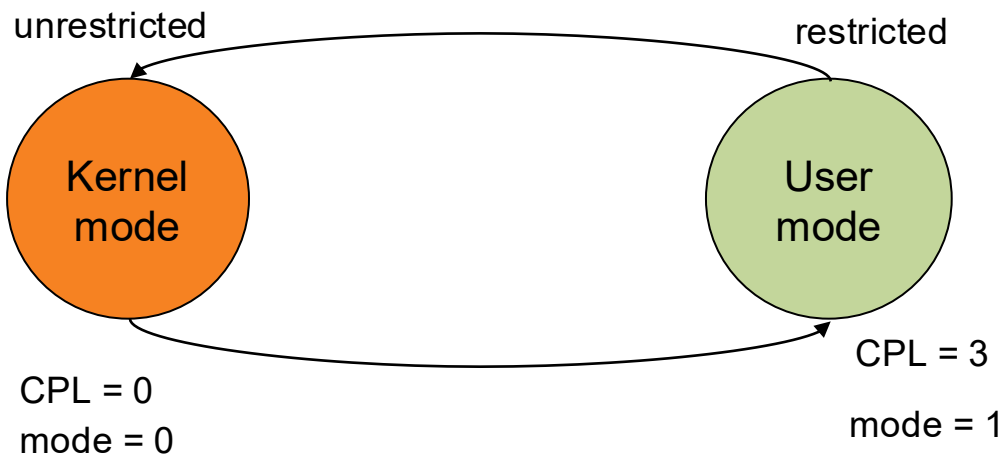
Privileged Instructions

- Instructions available in kernel mode, but not in user mode, are privileged instructions
  - Examples:
    - Change which memory locations a user program can access
    - Send commands to I/O devices
    - Read data from/write data to I/O devices
    - Disable interrupts
    - Jump into kernel code
- What should happen if a user program attempts to execute a privileged instruction?

©Tatyana Ryutov

Dual Mode Operation: HW Support

- Mode bit(s) in HW indicate the current mode
  - Stored in the processor status register
  - Not directly accessible to user code
- Examples:
  - Current Privilege Level (CPL) in x86 indicates current mode
    - CPL= 0 – kernel mode (ring 0), CPL= 3 – least privileged (ring 3)
  - Privileged/unprivileged modes in ARM7



©Tatyana Ryutov

ARM Modes

- ARM has seven basic operating modes
  - Each mode has access to it's own stack and a different set of registers
  - Sime operations can only be executed in a privileged mode

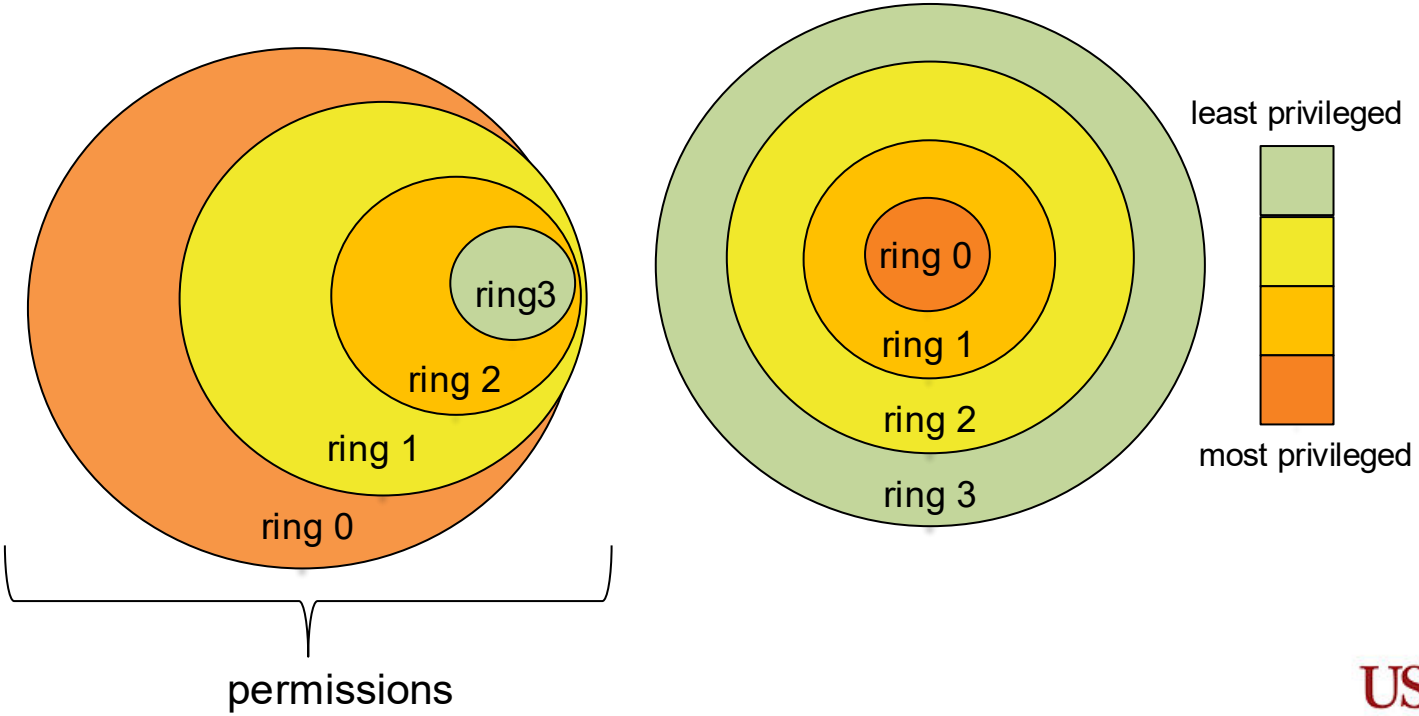
	Mode	Description	
Exception modes	Supervisor (SVC)	Entered on reset and when a Supervisor call instruction (SVC) is executed	Privileged modes
	FIQ	Entered when a high priority (fast) interrupt is raised	
	IRQ	Entered when a normal priority interrupt is raised	
	Abort	Used to handle memory access violations	
	Undef	Used to handle undefined instructions	
	System	Privileged mode using the same registers as User mode	
	User	Mode under which most Applications / OS tasks run	Unprivileged mode

More info: https://developer.arm.com/documentation/ddi0406/b/System-Level-Architecture/The-System-Level-Programmers--Model/ARM-processor-modes-and-core-registers/ARM-processor-modes?lang=en

©Tatyana Ryutov

HW Protection Rings

- Generalization of the user/kernel modes
- Define a fixed number N of hierarchical domains (called protection rings)
  - Named by the integers 0 through N – 1
  - Access capabilities available in ring m are a subset of those in ring n, m > n

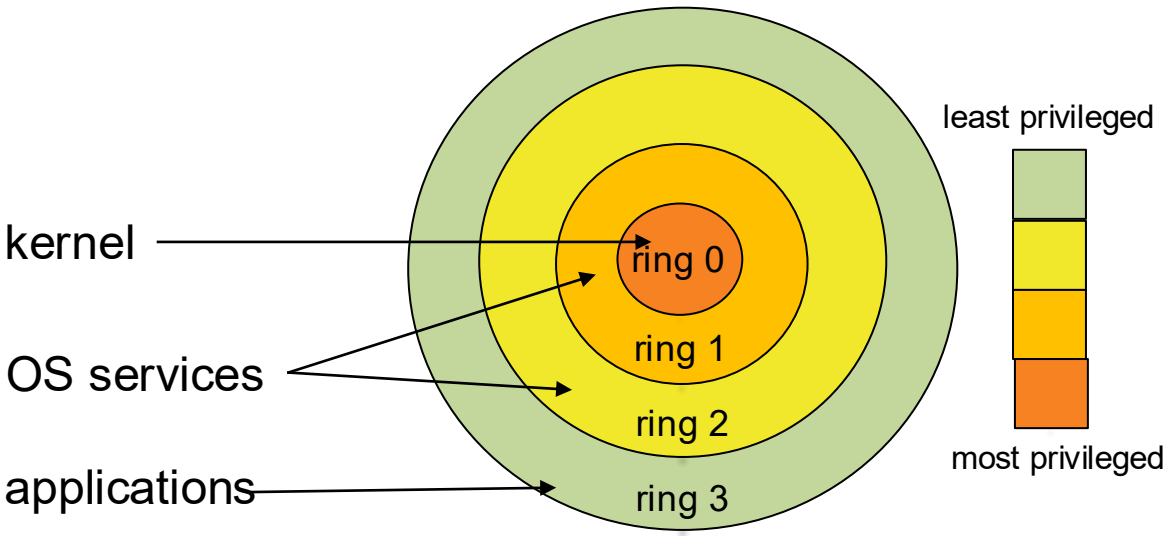


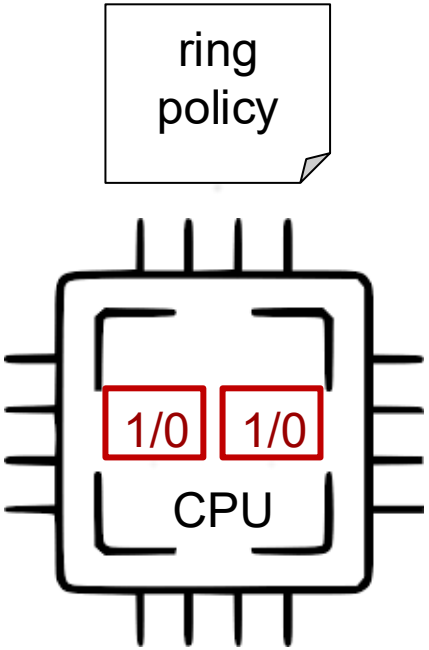
©Tatyana Ryutov

USC Viterbi School of Engineering

x86 HW Protection Rings

- Four levels of privilege (modes) can be interpreted as rings of protection
- Permissions are determined according to the ring policies
  - Ring 0 can do anything
  - Rings 1 and 2 are more limited than ring 0
  - Ring 3 can do nothing requiring privileged access





More info: Intel® 64 and IA-32 Architectures Software Developer's Manual,
See section 5.5 Privilege Levels and sections 5.6- 5.8
https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-vol-3a-part-1-manual.pdf

©Tatyana Ryutov

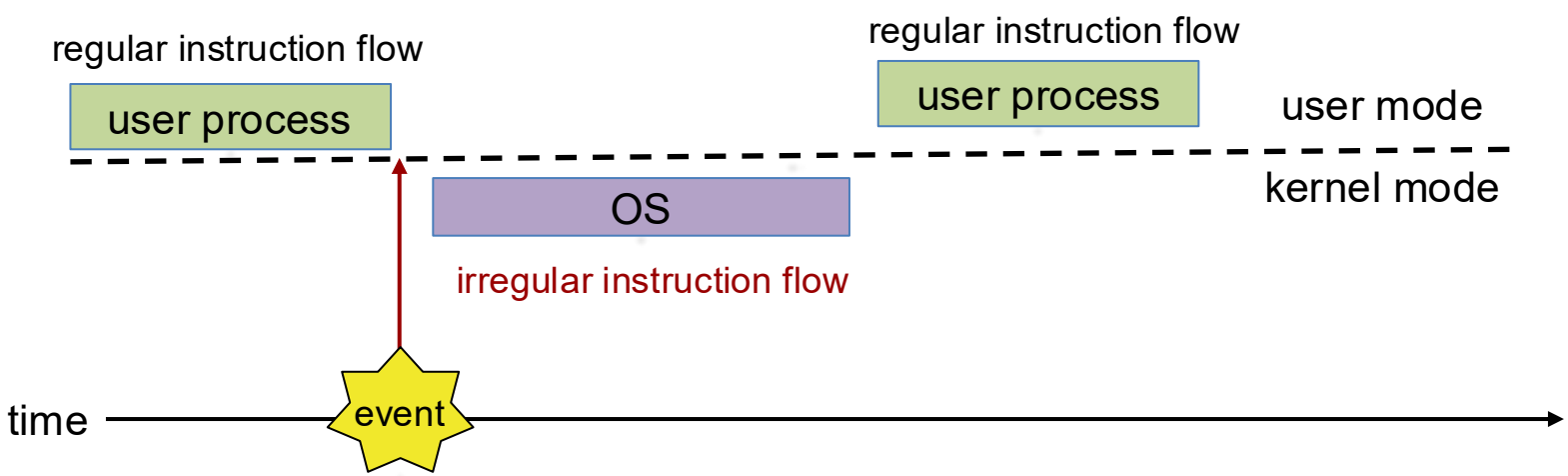
Lecture Outline

- Review
- Dual mode operation
- Event processing
- Context and process switch
- System calls
- Kernel organization

©Tatyana Ryutov

OS is Event Driven

- Event causes a break in normal execution flow, examples:
  - Error conditions/exceptions
  - HW interrupts raised by devices
  - System calls
- Events automatically cause transition to kernel mode
- When events occur, automatically call some subroutine (a.k.a. "handler") to handle the issue, then resume normal processing

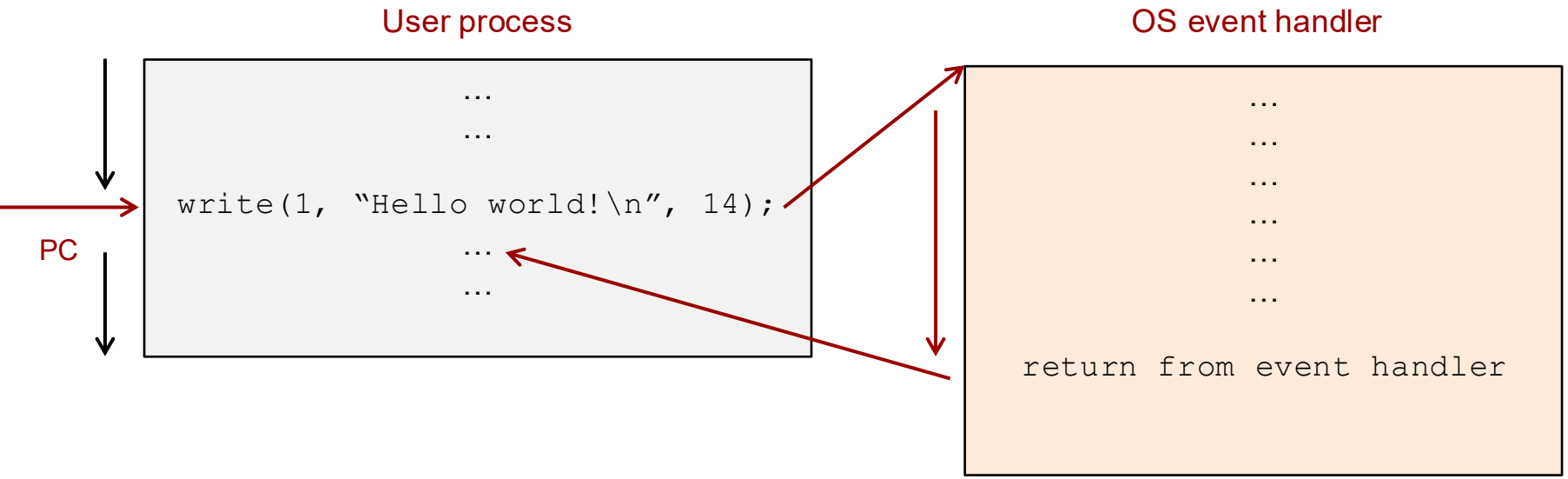


©Tatyana Ryutov

USC Viterbi School of Engineering

Event Processing

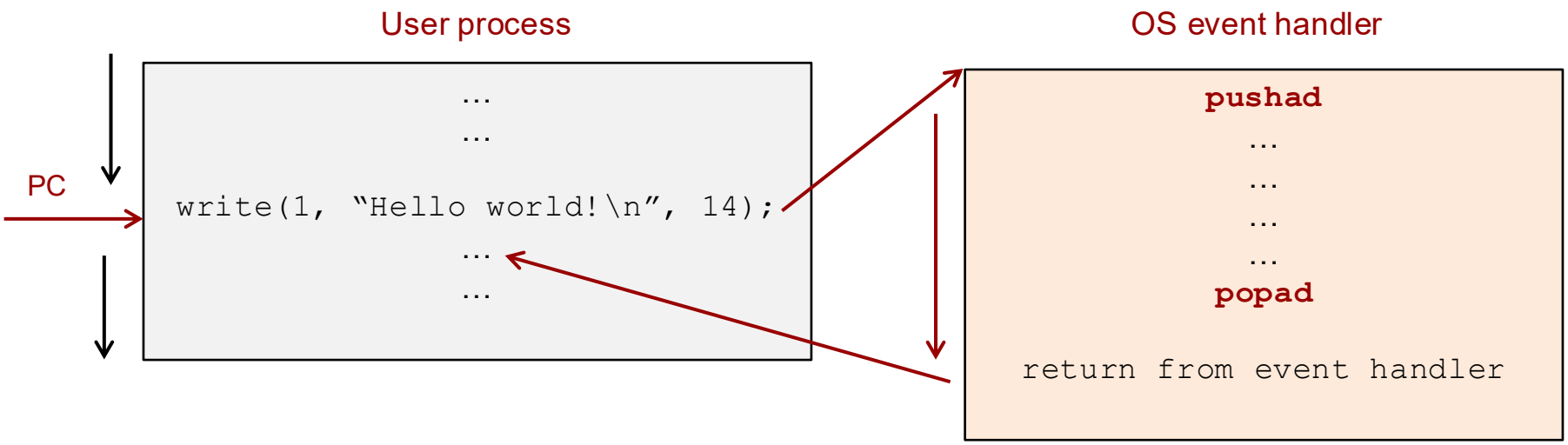
- What does the HW do when an event occurs?
  1. Save necessary CPU state to be able to restart the process
  2. Call appropriate “handler” routine to deal with the event
  3. Restore state and return to the process



©Tatyana Ryutov

Event Processing: Problem of Changed State

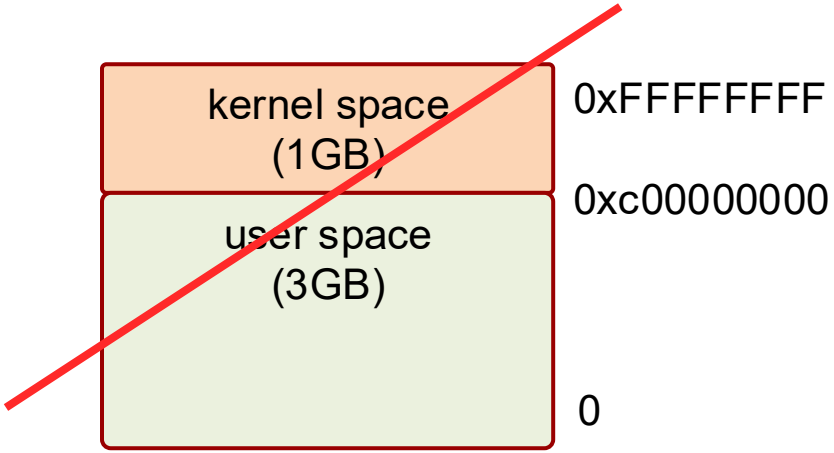
- Need to save current process registers without changing them
  - Not easy! Saving state needs to execute code, which will modify the registers
- Solution: Use hardware + software … architecture dependent
- Example
  - x86 architecture will save stack pointer, program counter (EIP), and EFLAGS register on the stack automatically when event occurs
  - Other registers must also be pushed onto the stack



©Tatyana Ryutov

Process's Double Life

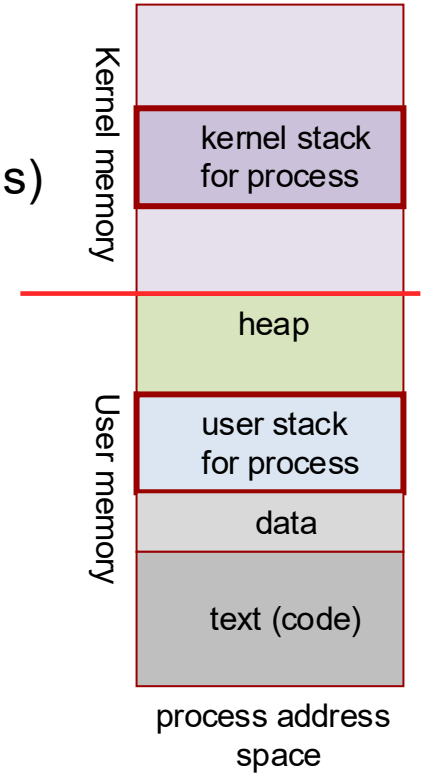
- We discussed user process as a user-mode-only concept
  - Time to correct that misconception!
- Each user process has separate user mode and kernel mode execution contexts:
  1. User-mode context: register values when running user code in user mode + user-mode stack
  2. Kernel-mode context: register values when running kernel code in kernel mode + kernel-mode stack
- So, a user process can "become" kernel when even occurs (e.g., syscall)



©Tatyana Ryutov

User and Kernel Stacks

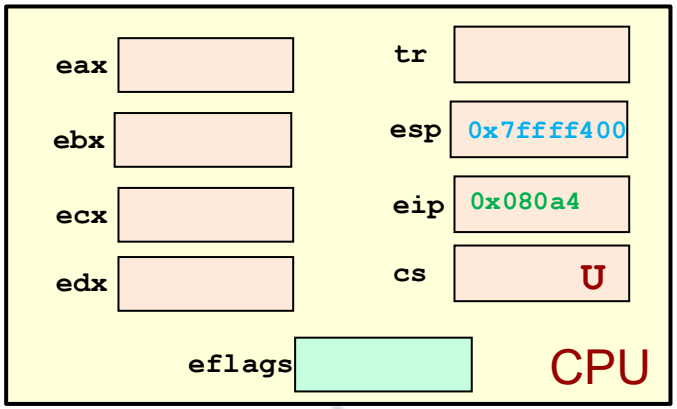
- In most OSs each user process has two stacks:
  - User stack
    - Used when executing user code
  - Kernel stack
    - Used when executing kernel code (e.g., during syscalls)
    - In xv6, **`p→kstack`** member of **`proc`** points to kernel stack
- HW changes the stack pointer to point to the base of the kernel stack on events, when privilege mode switches occur
  - Advantage: kernel can execute even if user stack is corrupted

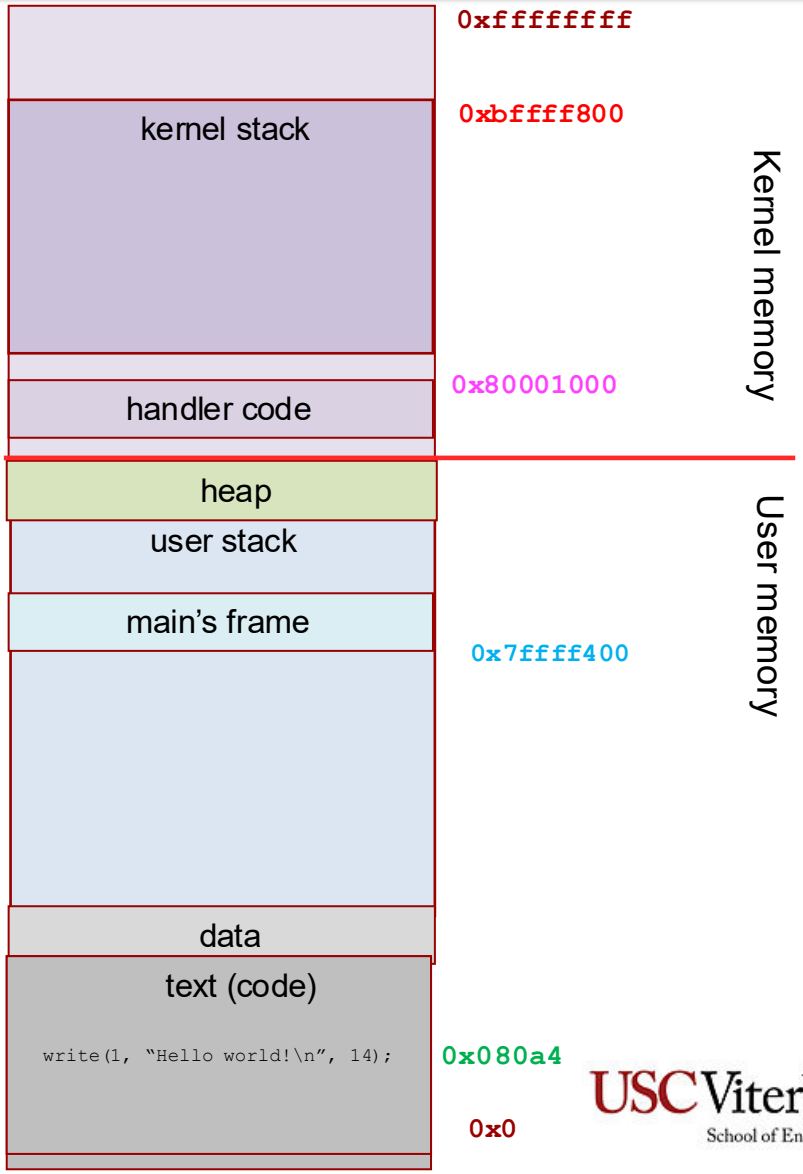


©Tatyana Ryutov

Transition from User to Kernel Mode

- User process is executing





©Tatyana Ryutov

USC Viterbi School of Engineering

Lecture Outline

- Review
- Dual mode operation
- Event processing
- Context and process switch
- System calls
- Kernel organization

©Tatyana Ryutov

Two Switches

- Recall that CPU has a state, which is the current contents of all of its registers
- Context switch – switching CPU from one running task to another by saving the CPU state to resume the task at a later time
- Process switch - switching CPU from one process to another by saving **all of the state** of the currently executing process

©Tatyana Ryutov

Note

- We need some basic x86 background
- Most of xv6 is in C, and assembly code (including all the stack manipulations for function calls)
- Small parts are in assembly language. Why?
- Basic understanding of x86 assembly language is required to follow some nuances of xv6 code

©Tatyana Ryutov

x86 Registers

- Small space for data storage within CPU
- General purpose registers: store data during computations (eax, ebx, ecx, edx, esi, edi)
- Pointers to stack locations: base of stack (ebp) and top of stack (esp)
- Program counter or instruction pointer (eip): next instruction to execute
- Control registers: hold control information or metadata of a process (e.g., cr3 has pointer to page table)
- Segment registers (cs, ds, es, fs, gs, ss): information about segments (related to memory of process)

When process calls a function, who saves the registers?

©Tatyana Ryutov

32 bit x86 C Calling Conventions

- Caller is a function that calls another function
- Callee is a function that was called
- Caller responsibilities:
  - Save content of caller-saved registers: EAX, ECX, EDX
  - Push the function arguments on the stack
  - Call the function (use **call** instruction)
  - Restore the contents of caller-saved registers and arguments to the call from stack
- Callee responsibilities:
  - Push the value of EBP onto the stack, and then copy the value of ESP into EBP
  - Save the content of callee-saved registers: EBX, EDI and ESI
  - Allocate local variables and perform computation
  - When the callee is done, it does the following to return:
    - Store the return value in the %eax register
    - Restore the old values of any callee-saved registers
    - Restore the caller's base pointer value by popping EBP off the stack
    - Return control to the caller via the **ret** instruction, which pops the return address off the stack and jumps there

©Tatyana Ryutov

xv6 trapframe

- **`struct trapframe`** contains register state stored on the kernel stack of a process

```
struct trapframe {
  // registers as pushed by pusha
  uint edi;
  uint esi;
  uint ebp;
  uint ebx;
  uint edx;
  uint ecx;
  uint eax;

  // rest of trap frame
  ushort gs;
  ushort padding1;
  ushort fs;
  ushort padding2;
  ushort es;
  ushort padding3;
  ushort ds;
  ushort padding4;
  uint trapno;

  // below here defined by x86 hardware
  uint err;
  uint eip;
  ushort cs;
  ushort padding5;
  uint eflags;

  // below here only when crossing rings, such as from user to kernel
  uint esp;
  ushort ss;
  ushort padding6;
};
```

stores context when switching to the kernel mode

x86.h

```
struct proc {
  uint sz;
  pde_t* pgdir;
  char *kstack;
  enum procstate state;
  int pid;
  struct proc *parent;
  struct trapframe *tf;
  struct context *context;
  void *chan;
  int killed;
  struct file *ofile[NOFILE];
  struct inode *cwd;
  char name[16];
};
```

proc.h

©Tatyana Ryutov

xv.6 Context Structure

- **`struct context`** holds set of registers to be saved when switching from one process to another
- Context is pushed on to the kernel stack of the process

```
struct context {
   uint edi;
   uint esi;
   uint ebx;
   uint ebp;
   uint eip;
};                    proc.h
```

```
struct proc {
  uint sz;
  pde_t* pgdir;
  char *kstack;
  enum procstate state;
  int pid;
  struct proc *parent;
  struct trapframe *tf;
  struct context *context;
  void *chan;
  int killed;
  struct file *ofile[NOFILE];
  struct inode *cwd;
  char name[16];
};
```

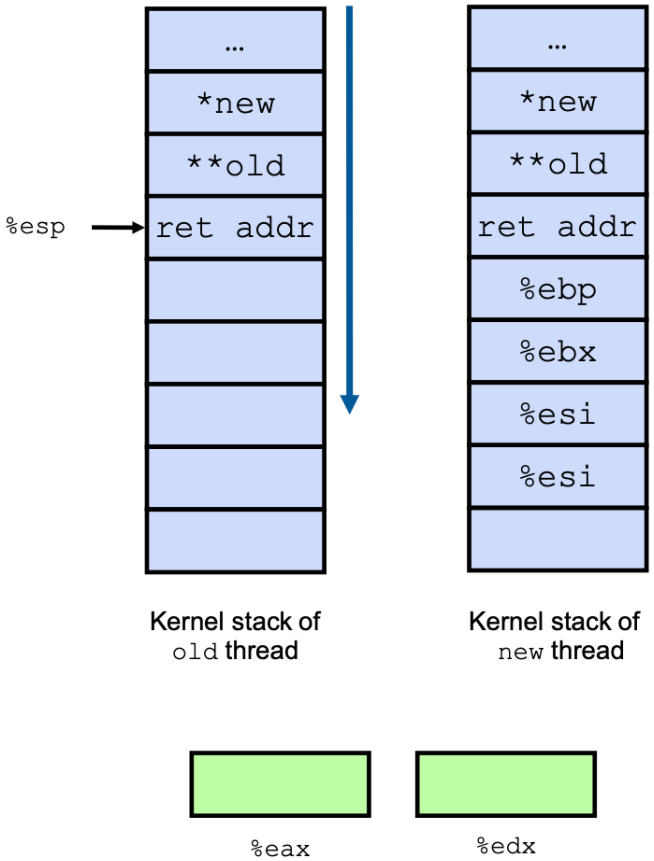
©Tatyana Ryutov

xv6: Context Switching

- **`swtch`** function switches between two contexts, an old one and a new one

```
swtch(struct context **old, struct context *new)

# load arguments to %eax and %edx
movl 4(%esp), %eax //old
movl 8(%esp), %edx //new

 # Pushes the registers (Save context of old)
pushl %ebp
pushl %ebx
pushl %esi
pushl %edi

 # Switch stack.
movl %esp, (%eax) //Saves %esp at *old
movl %edx, %esp   //Restore %esp from new

# Load registers (Restore context of new)
popl %edi
popl %esi
popl %ebx
popl %ebp

# make return address as %eip
ret
```

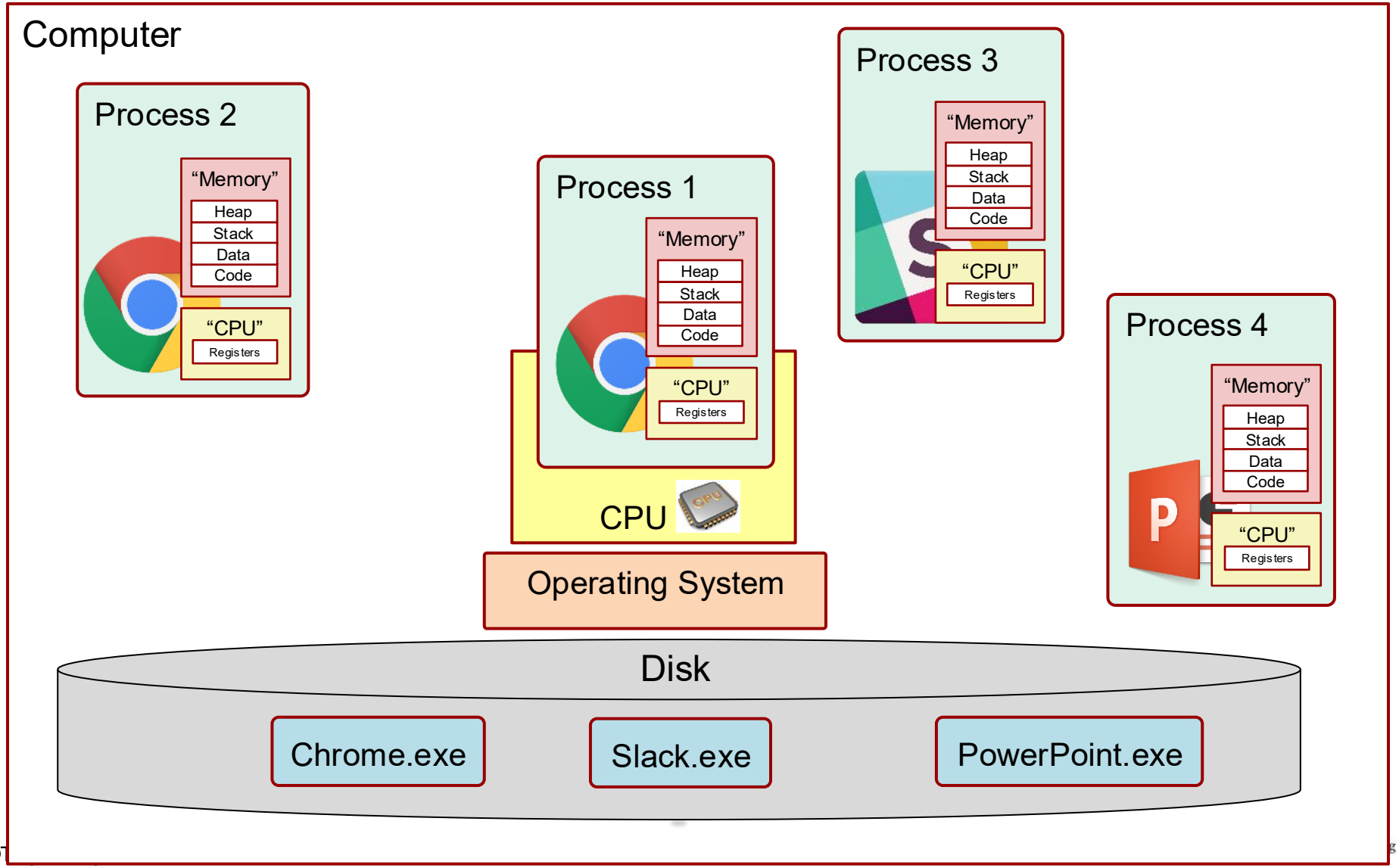
swtch.S



©Tatyana Ryutov

USC Viterbi School of Engineering

Process Switch: Intuition

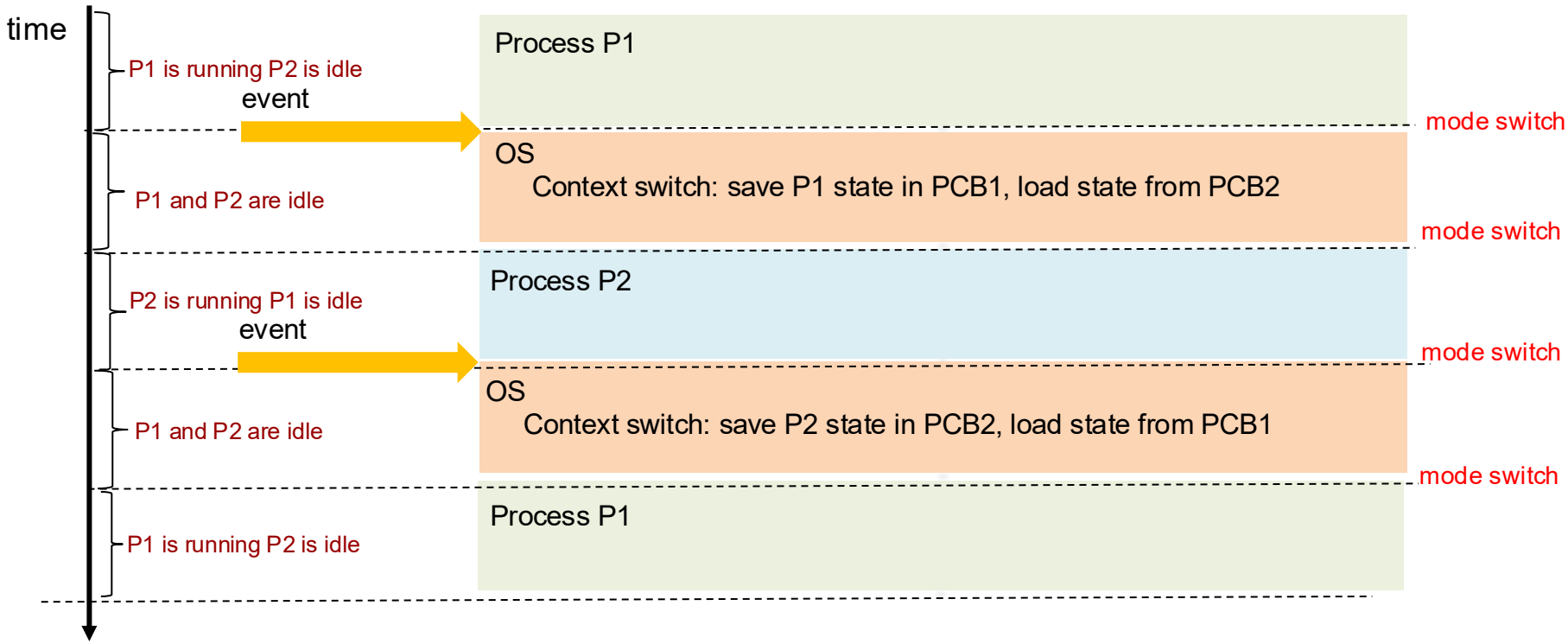


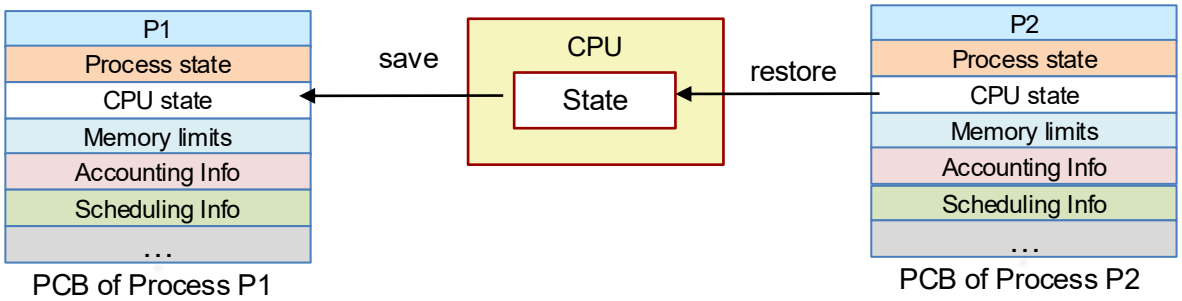
Process Switch

- Process switches are performed by the OS to stop executing a running process and begin executing a new ready process
- How to perform a process switch?
  1. Save current process state
  2. Load state of the next ready process
  3. Continue execution of the next process

©Tatyana Ryutov

Process Switch on a Uniprocessor





©Tatyana Ryutov

USC Viterbi School of Engineering

xv6 Scheduler

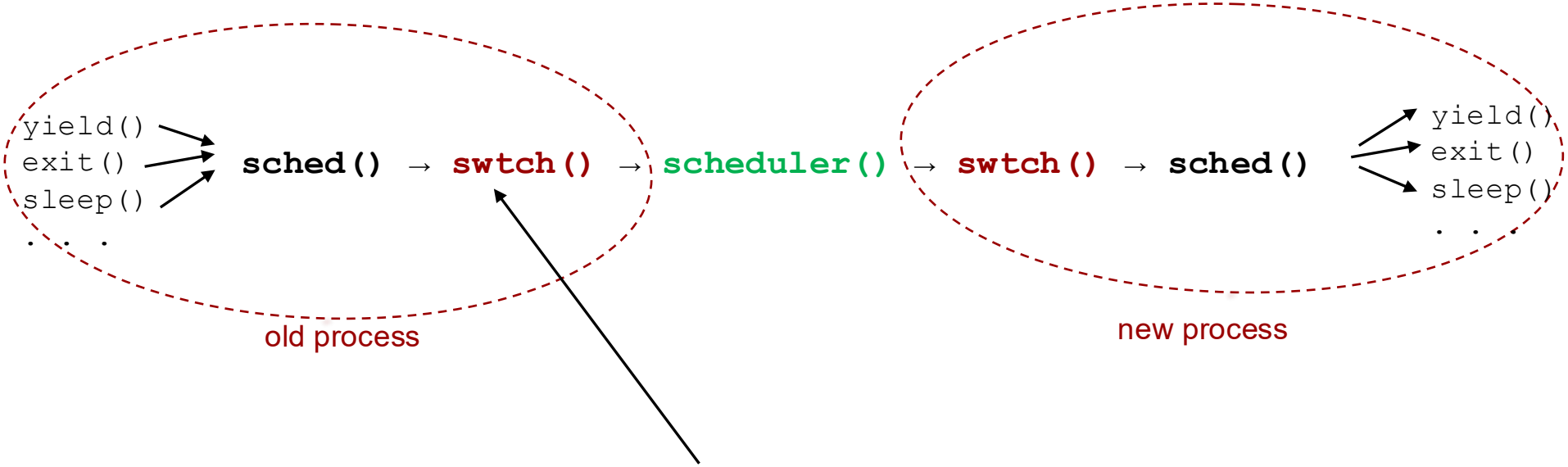
- Each CPU has a scheduler thread that calls the `scheduler` function
- Scheduler goes over list of processes and switches to a runnable one
- After running for some time, the process switches back to the scheduler thread, when:
  - Process terminated
  - Process needs to sleep (e.g., blocking syscall)
  - Process gives up CPU (e.g., timer interrupt)
- Scheduler thread runs its loop and picks next process to run, and the story repeats

```
struct {
    struct spinlock loc;
    struct proc proc[NPROC];
} ptable;
```

©Tatyana Ryutov

xv6 Process Switching Overview

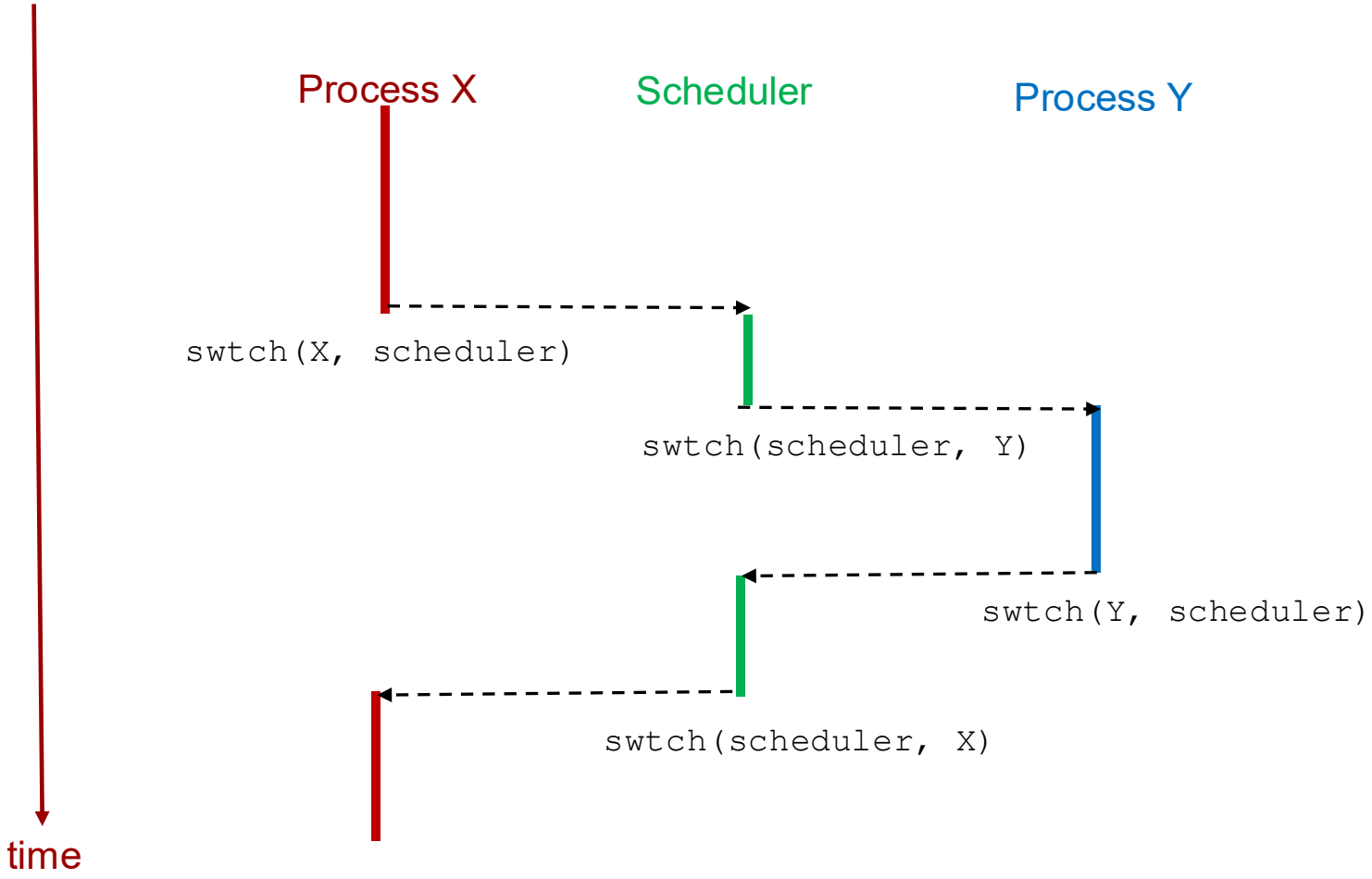
- xv6 never directly switches from one user process to another
- Scheduler runs as a separate thread with its own stack



- A running process always gives up CPU at this call to swtch, and always resumes execution at this point at a later time
- The only exception is …

©Tatyana Ryutov

xv6 Process Switching Timeline



©Tatyana Ryutov

USC Viterbi School of Engineering

shed and sheduler

- User process switches to scheduler thread in the “sched” function (invoked from exit, sleep, yield)
- Scheduler switches to user process in “scheduler” function

```
void sched(void) {
int intena; struct proc *p = myproc();

if(!holding(&ptable.lock)) // make sure the ptable is locked.
 panic("sched ptable.lock");

...

swtch(&p->context, mycpu()->scheduler);
...
}
```

proc.c

```
void scheduler(void) {
...
for(;;){

...
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
if(p->state != RUNNABLE)
 continue;
c->proc = p;
switchuvm(p);
p->state = RUNNING;

swtch(&(c->scheduler), p->context);
switchkvm();
  ...
  }
...
}
```

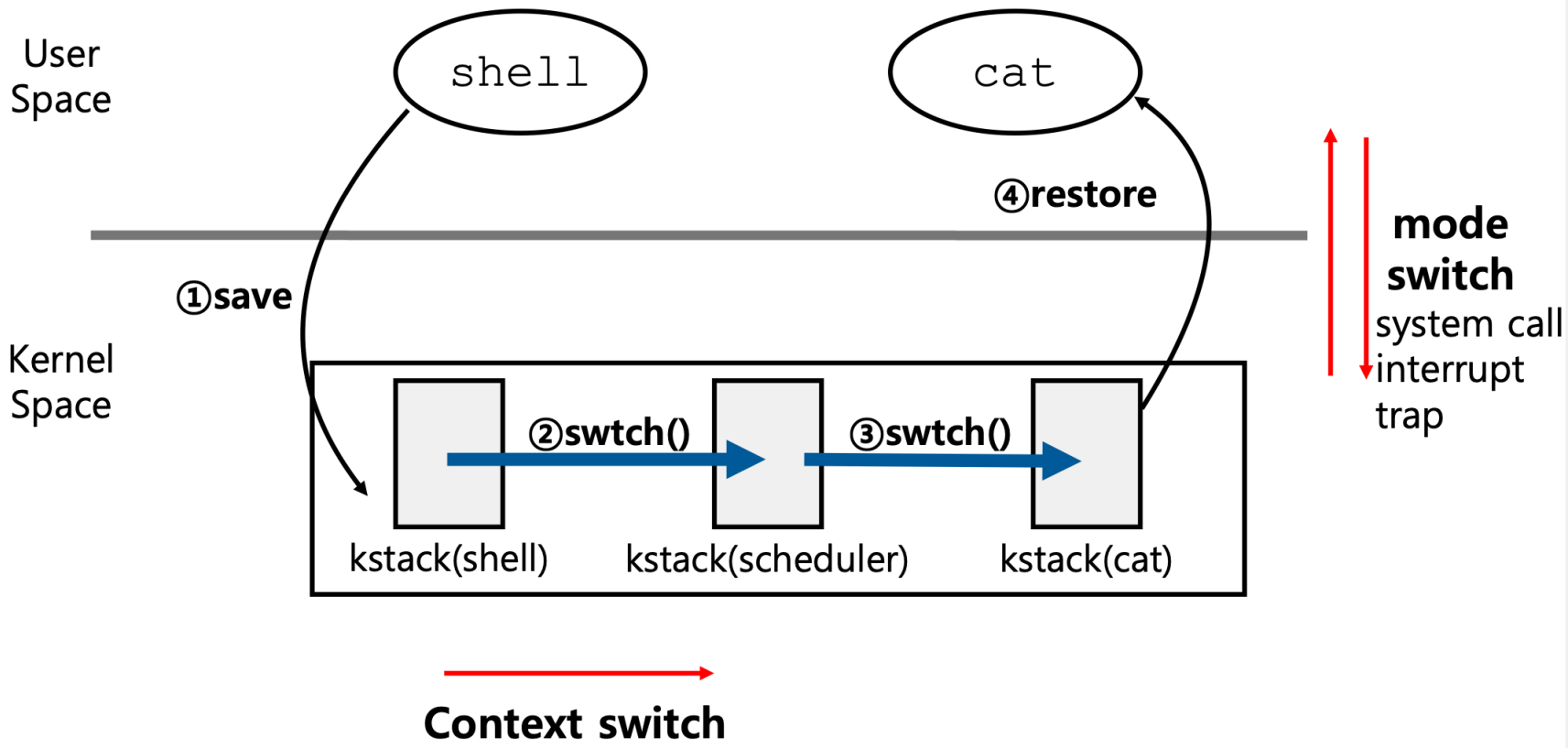
Switch to the address space of user process

Processor switches to execute the process

Process comes back to the scheduler, switch to the address space of the kernel

©Tatyana Ryutov

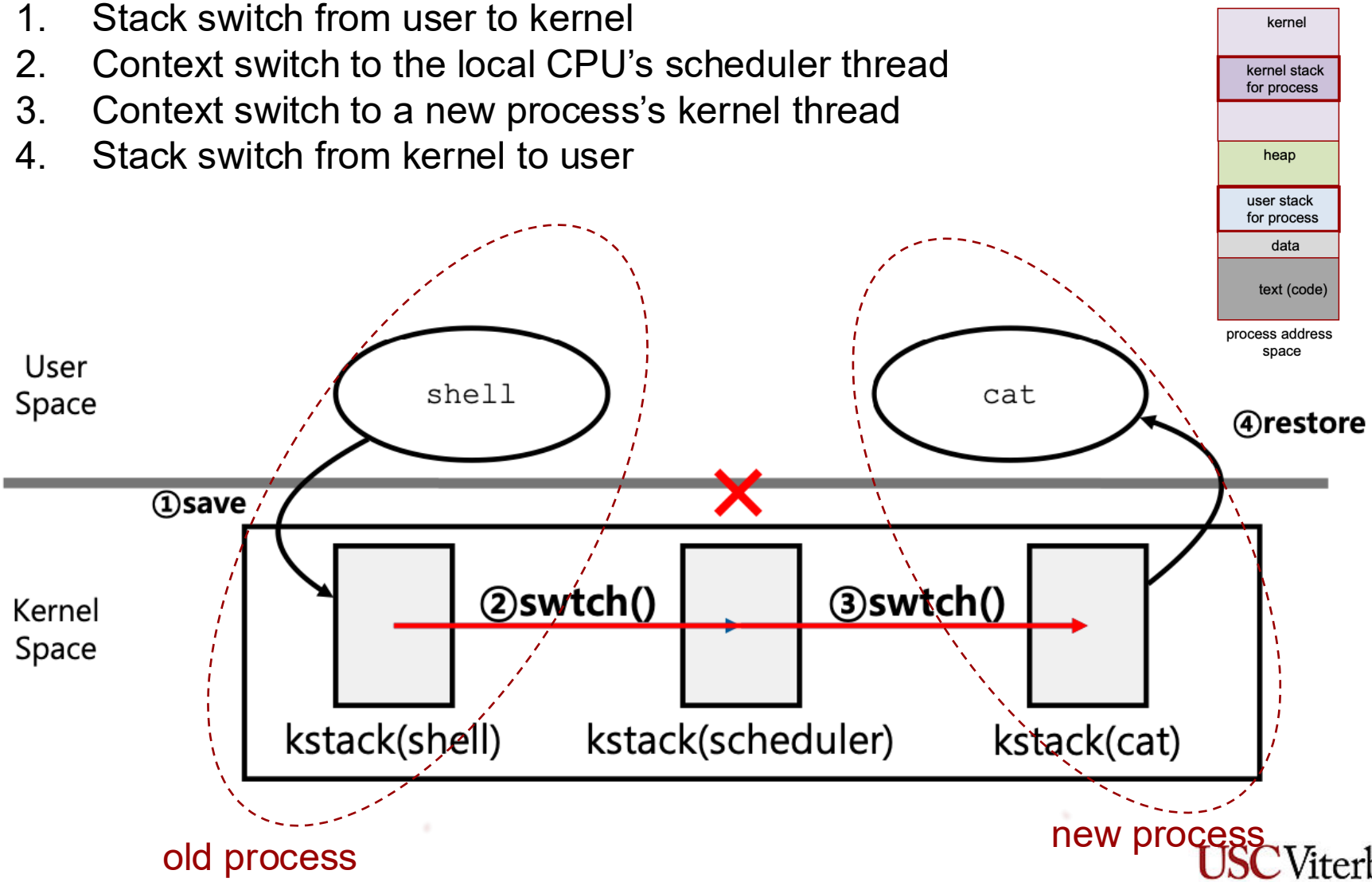
xv6 Process Switching Example



©Tatyana Ryutov

x.v6 Process Switch Example

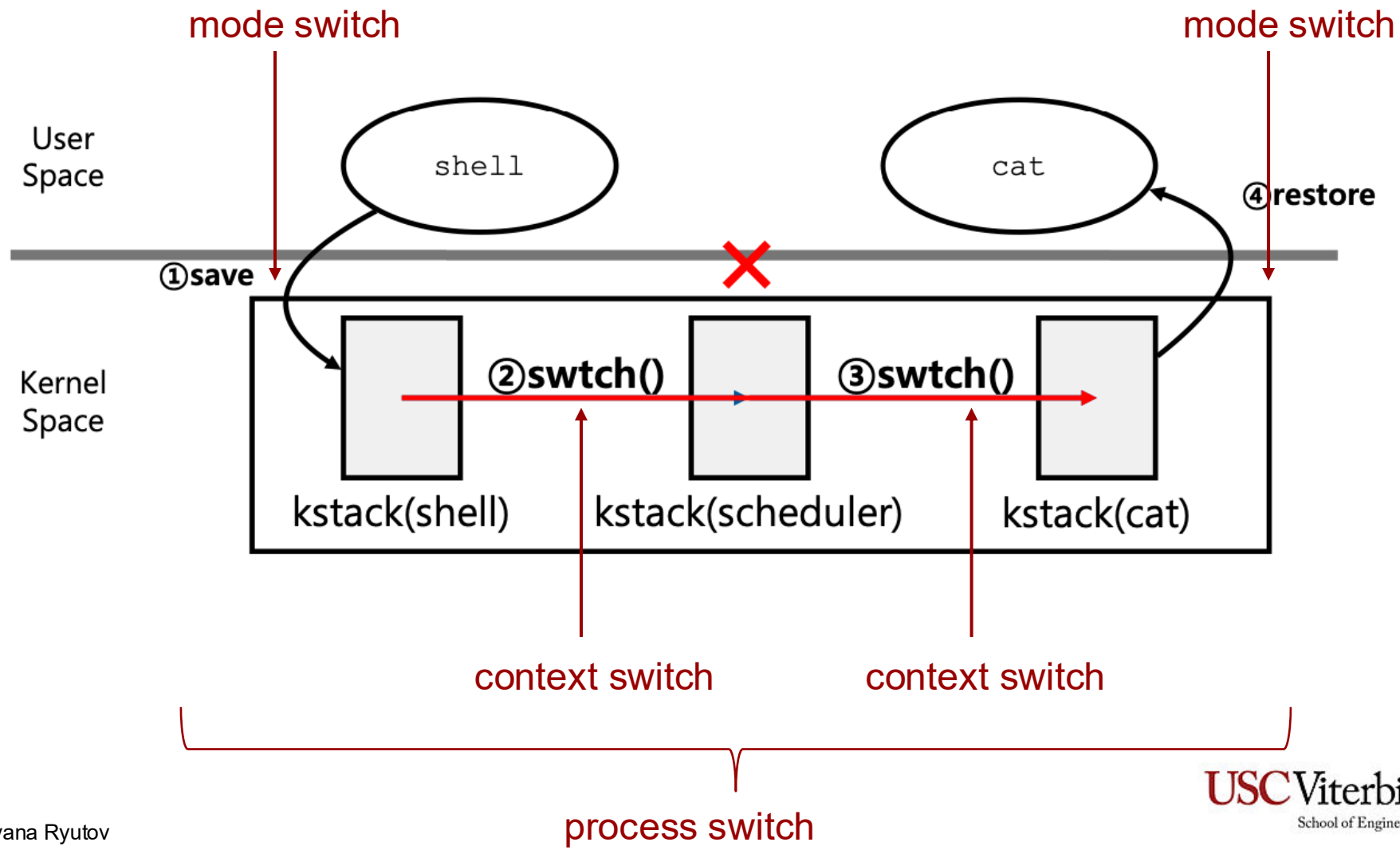
1. Stack switch from user to kernel
2. Context switch to the local CPU's scheduler thread
3. Context switch to a new process's kernel thread
4. Stack switch from kernel to user



©Tatyana Ryutov

USC Viterbi School of Engineering

x.v6 Process Switch Example



©Tatyana Ryutov

USC Viterbi School of Engineering

xv6 context structure vs. trapframe

- `context` and `trapframe` structures contain register state stored on kernel stack of a process. What is the difference?

```
struct trapframe {
  // registers as pushed by pusha
  uint edi;
  uint esi;
  uint ebp;
  uint ebx;
  uint edx;
  uint ecx;
  uint eax;

  // rest of trap frame
  ushort gs;
  ushort padding1;
  ushort fs;
  ushort padding2;
  ushort es;
  ushort padding3;
  ushort ds;
  ushort padding4;
  uint trapno;

  // below here defined by x86 hardware
  uint err;
  uint eip;
  ushort cs;
  ushort padding5;
  uint eflags;

  // below here only when crossing rings, such as from user to
kernel
  uint esp;
  ushort ss;
  ushort padding6;
};
```

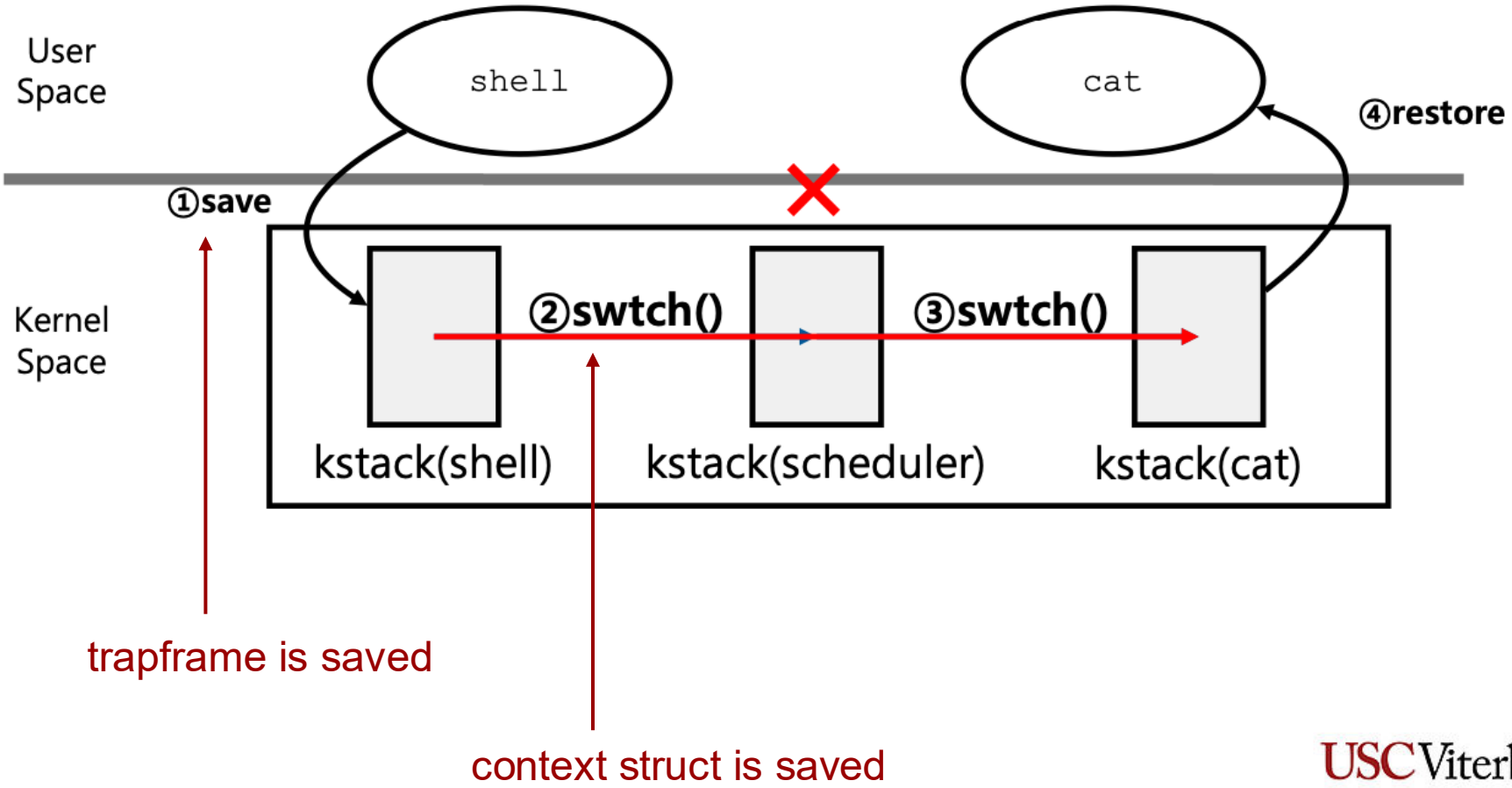
```
struct proc {
  uint sz;
  pde_t* pgdir;
  char *kstack;
  enum procstate state;
  int pid;
  struct proc *parent;
  struct trapframe *tf;
  struct context *context;
  void *chan;
  int killed;
  struct file *ofile[NOFILE];
  struct inode *cwd;
  char name[16];
};
```

```
struct context {
   uint edi;
   uint esi;
   uint ebx;
   uint ebp;
   uint eip;
};
```

©Tatyana Ryutov

Example

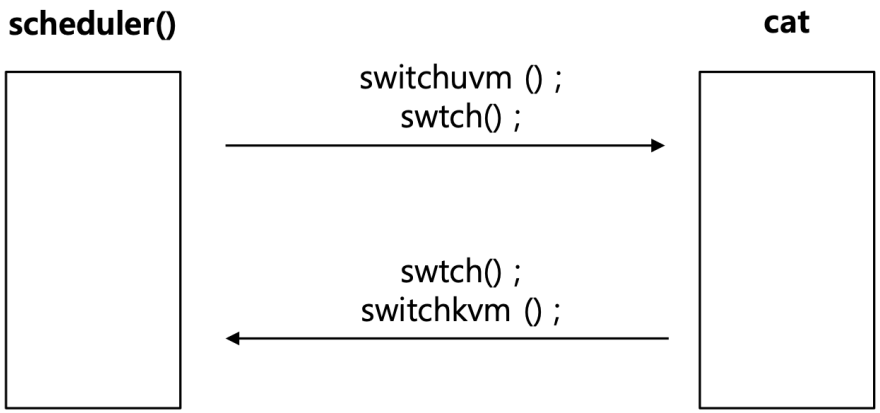
- Example: user process has timer interrupt, saves trapframe on kstack, then calls swtch, saves context structure on kstack



©Tatyana Ryutov

Missing Pieces

- Showed how we change kernel registers, stacks, program counter, this is not the whole story
- Saving user-space part?
  - User registers stored in **`trapframe`** struct created on the kernel stack when event happens and restored before using iret to switch to user mode
- Switching address spaces?
  - `scheduler()` is responsible for switching the address spaces by calling **`switchuvm`**



©Tatyana Ryutov

Summary of Switching in xv6

- What happens during context switching?
  - Save register state (CPU context) on kernel stack of old process
  - Update context structure pointer of old process to this saved context
  - Switch from old kernel stack to new kernel stack
  - Restore register state (CPU context) from new kernel stack, and resume new process
- What happens during process switching (e.g., P1 to P2)?
  - P1 goes to kernel mode and gives up CPU (timer interrupt or exit or sleep)
  - P2 is another process that is ready to run (it had given up CPU after saving context on its kernel stack)
  - P1 switches to CPU scheduler thread
  - Scheduler thread finds runnable process P2 and switches to it
  - P2 returns to user mode

©Tatyana Ryutov

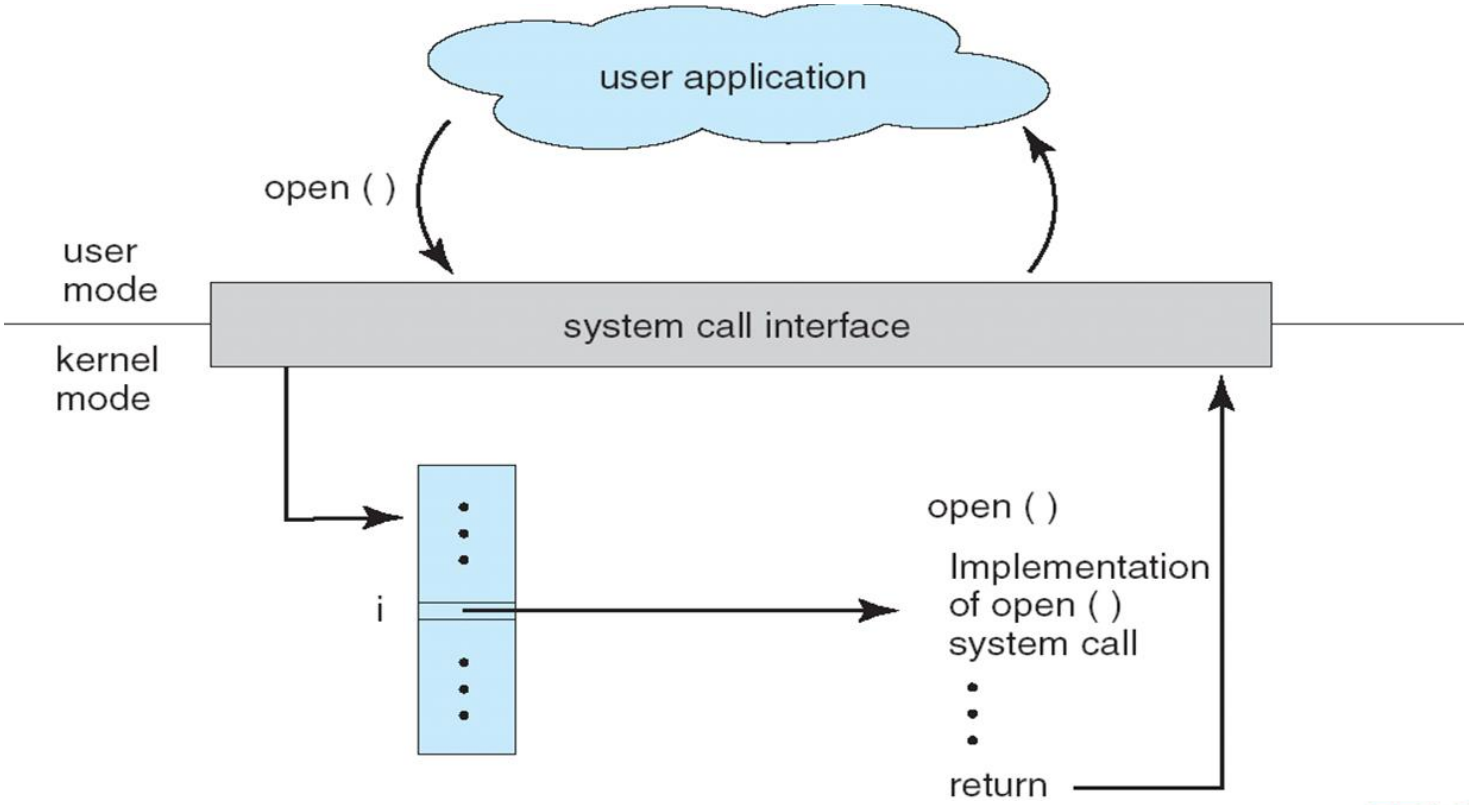
Lecture Outline

- Review
- Dual mode operation
- Event processing
- Context and process switch
- System calls
- Kernel organization

©Tatyana Ryutov

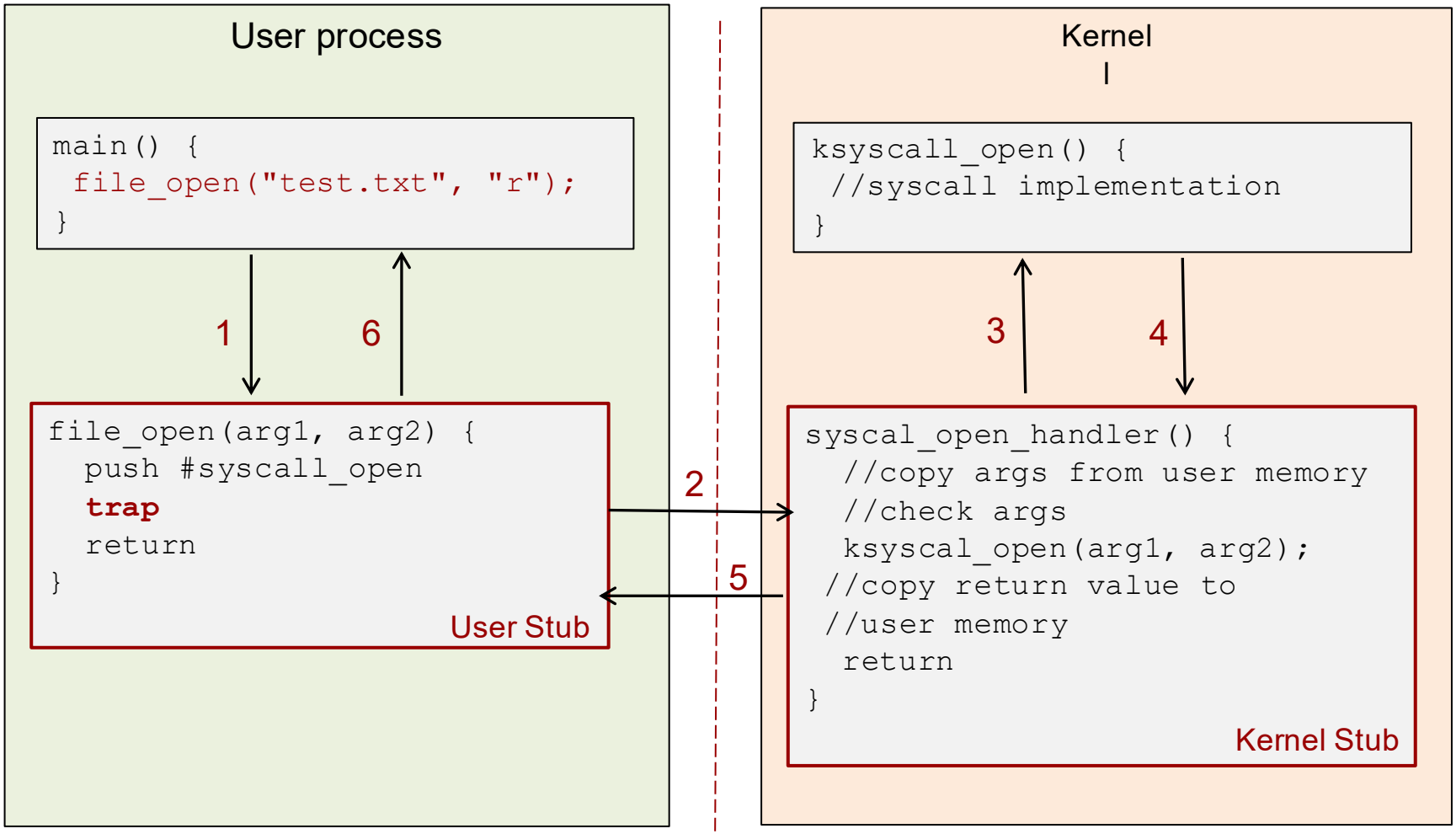
Recall: System Calls

- System Call is an interface between a user apps and services provided by OS
  - Provides a safe mechanism to request specific kernel operations
  - The API of the OS from the user program's point of view, accessible via a library of code, e.g., **`libc`**



©Tatyana Ryutov

Syscall Processing Overview



User-mode execution
(code from program ELF)

Kernel-mode execution
(code from kernel binary)

©Tatyana Ryutov

USC Viterbi School of Engineering

User and Kernel Syscall Handlers

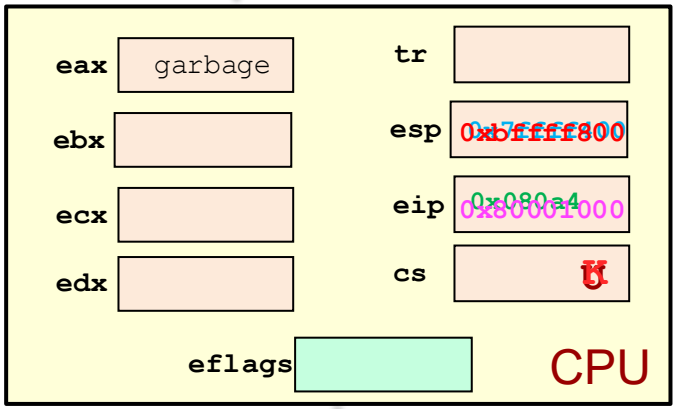
- User stub is responsible for:
  - Filling in the syscall number for the syscall
  - Pushing the arguments and the syscall number on the user stack (or registers)
  - Executing the trap instruction
- Kernel stub is responsible for:
  - Locating arguments
  - Copying arguments from user memory into kernel memory
  - Validating arguments
  - Copying results back into user memory

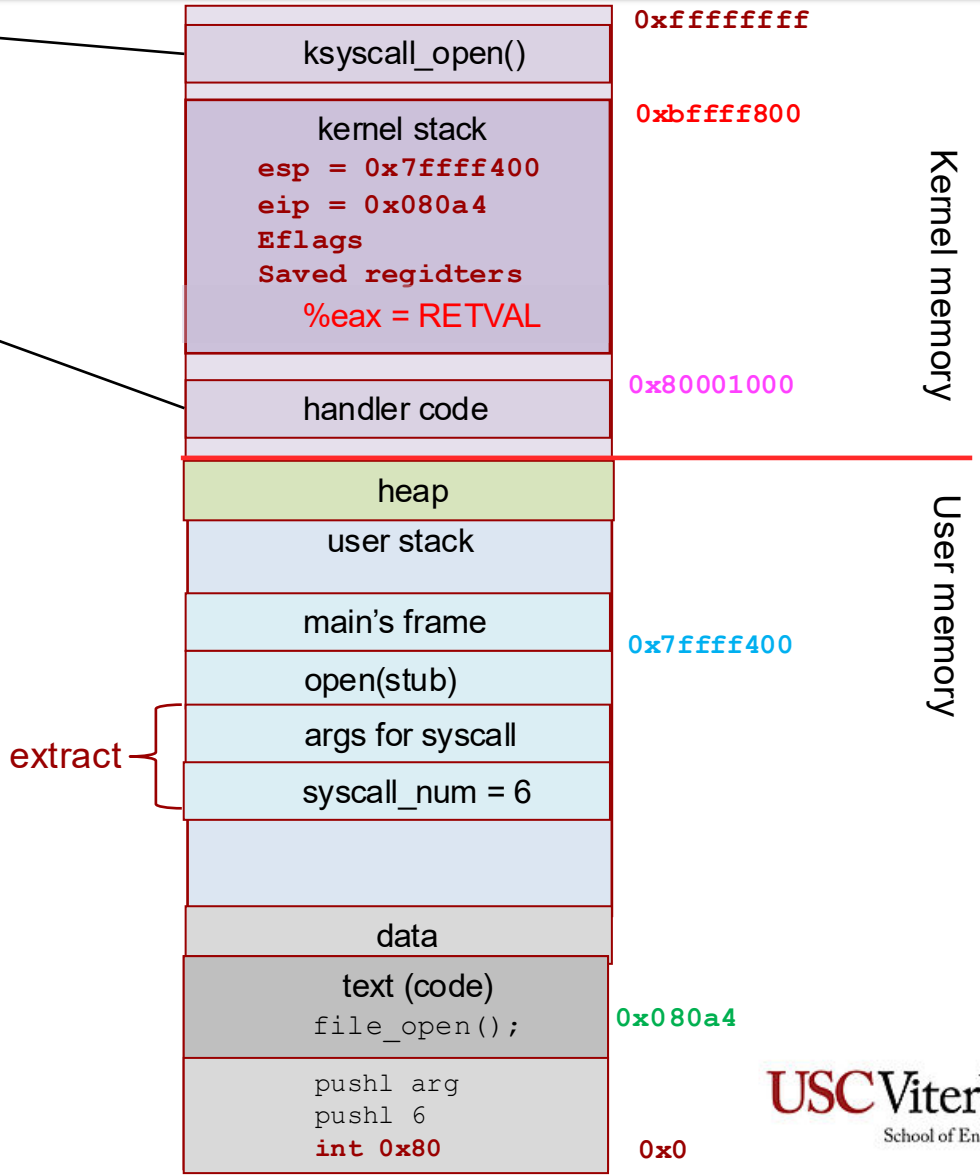
©Tatyana Ryutov

Syscall Execution Example

```
int ksyscall_open(…)
{
  //Extract arguments
  //Perform computation
  return value;
}
```

```
HANDLER30:
{
  pushad
  //Extract syscall num
  //If num == 0 …
  //If num = 1 …
  //...
  //If num == 6
  call ksyscall_open
  //Store rectal in %eax
   ...
   popad
  iret
}
```

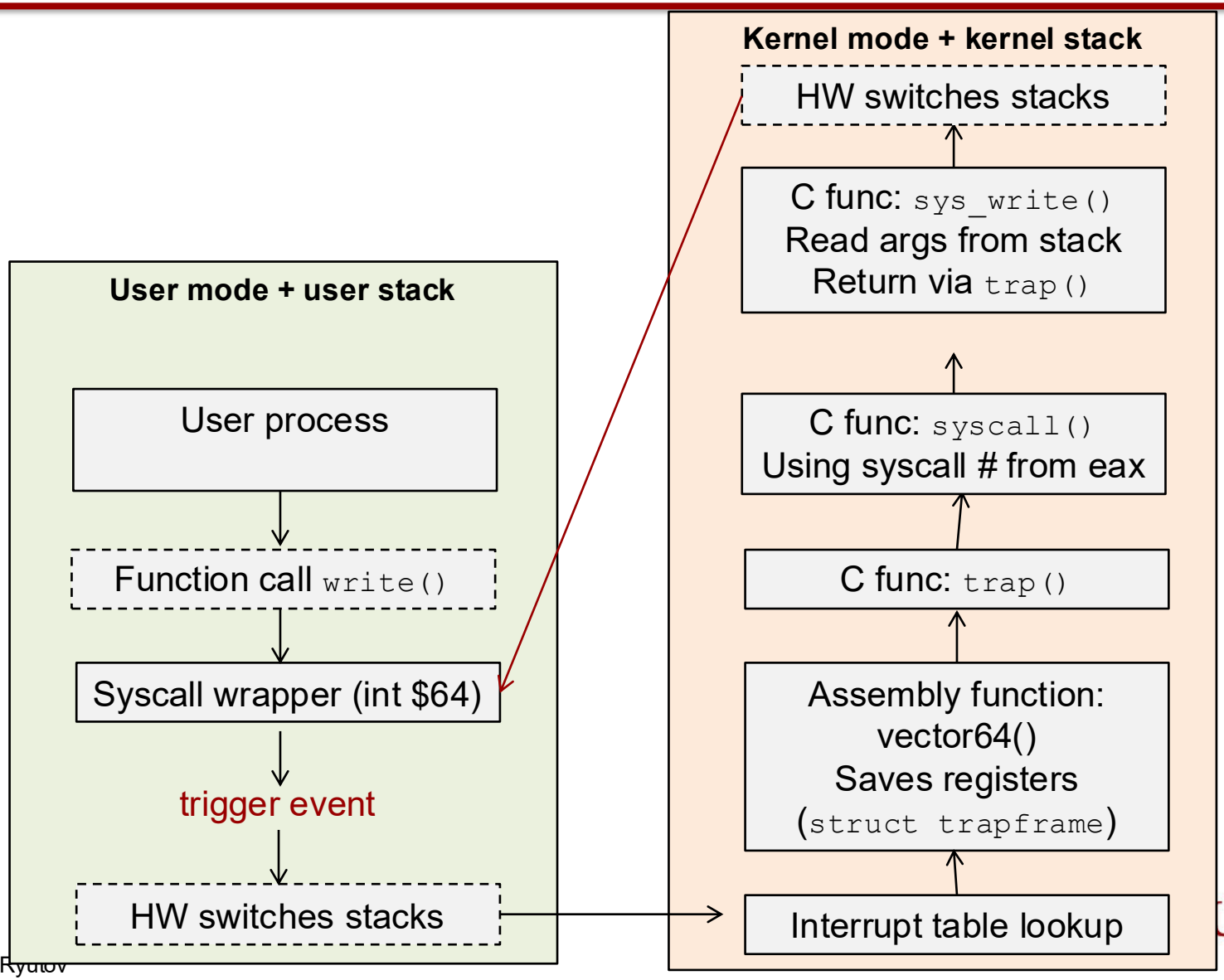




©Tatyana Ryutov

USC Viterbi School of Engineering

Syscall in xv6



©Tatyana Rydlov

USC Viterbi School of Engineering

xv6 write syscall: User Side

```
...
write(1, "Hello world!\n", 14);
...
```

main.c

```
...
#define SYS_write 16
...
```

syscal.h

```
...
#define T_SYSCALL 64
...
```

traps.h

(partial, after macro replacement)

```
.globl write
write:
    movl $SYS_write, %eax
    int $T_SYSCALL
    ret
```

usys.S

interrupt — trigger an event (64 in this case)

xv6 syscall calling convention: %eax = syscall number
otherwise: same as 32-bit x86 calling convention (arguments on stack)

©Tatyana Ryutov

xv6 write syscall: Kernel Side

```
void trap(struct trapframe *tf)
{
   if(tf->trapno == T_SYSCALL){
     if(myproc()->killed)
        exit();

     myproc()->tf = tf;

     syscall();

     if(myproc()->killed)
       exit();
   return;
  }
...
}
```

trap.c

`struct trapframe` — set by assembly: interrupt type, application registers, ... example: tf→eax = old value of eax

`myproc()` — represents currently running process

`syscall()` — actual implementation uses `myproc()->tf` to determine what to do

©Tatyana Ryutov

xv6 write syscall: Kernel Side

```
static int (*syscalls[])(void) = { ...
[SYS_write] sys_write,
...
};
...
void syscall(void) {
...
  num = curproc->tf->eax;
   if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
   curproc->tf->eax = syscalls[num]();  } else {
...
}
```

syscall.c

`syscalls []` array of functions — one for syscall '[number] value': syscalls[number] = value

(if system call number in range) call sys_...function from table store result in user's eax register

result assigned to eax (assembly code this returns to copies tf−>eax into %eax)

©Tatyana Ryutov

xv6 write syscall: Kernel Side

```
int sys_write(void)
{
  struct file *f;
  int n;
  char *p;

  if(argfd(0, 0, &f) < 0 || argint(2, &n) < 0 || argptr(1, &p, n) < 0)
     return -1;
  return filewrite(f, p, n);
}
```

sysfile.c

utility functions that read arguments from user's stack returns -1 on error

actual internal function that implements writing to a file

when done, trap returns to `alltraps` that restores registers from tf, then returns to user-mode

©Tatyana Ryutov

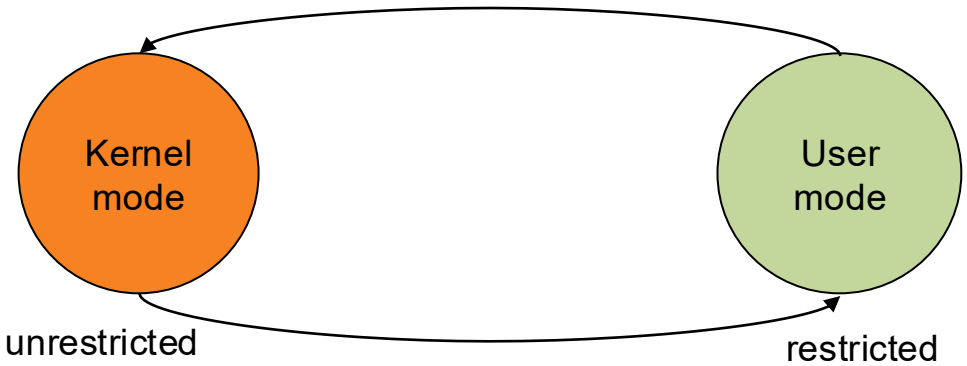
Lecture Outline

- Review
- Dual mode operation
- Event processing
- Context and process switch
- System calls
- Kernel organization

©Tatyana Ryutov

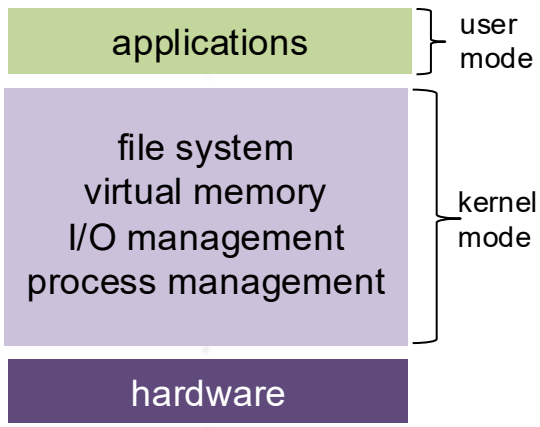
Kernel Organization

- OS must provide many services to user processes
- Key design question: where to put those services (user or kernel mode)?
- Tradeoffs: evaluate where the services should be located based on
  1. Performance
  2. Reliability/Security
  3. Flexibility/Comprehensibility
- Kernel organization approaches:
  - Monolithic
    - e.g., Windows, Linux, Mac OS
  - Layered
    - e.g., Multics, Gemsos
  - Microkernel
    - e.g., Mach, seL4
  - Virtual machine
    - e.g., Xen, VMware

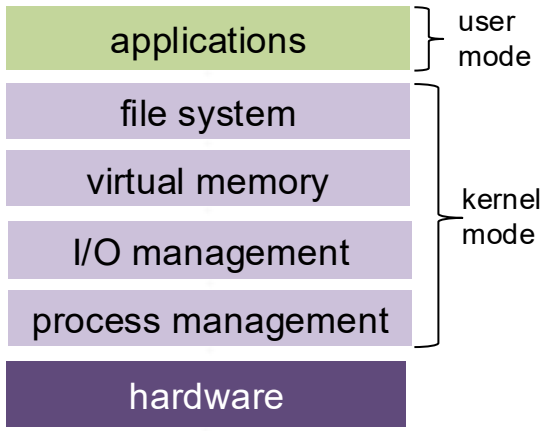


©Tatyana Ryutov

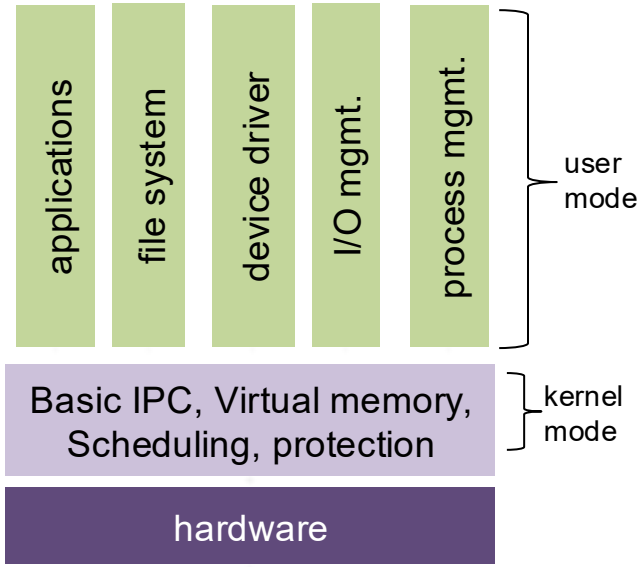
Kernel Organization Approaches



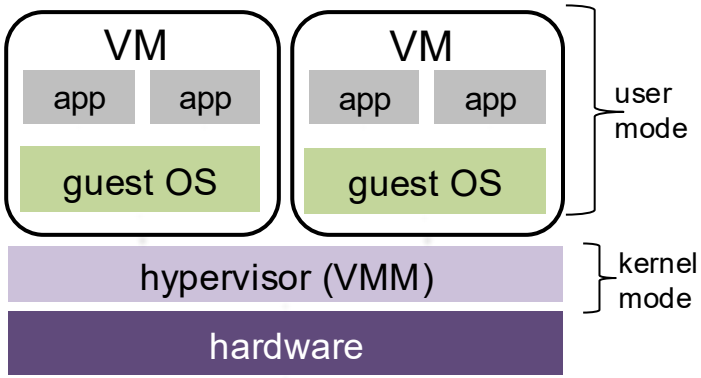
Monolithic



Layered



Microkernel

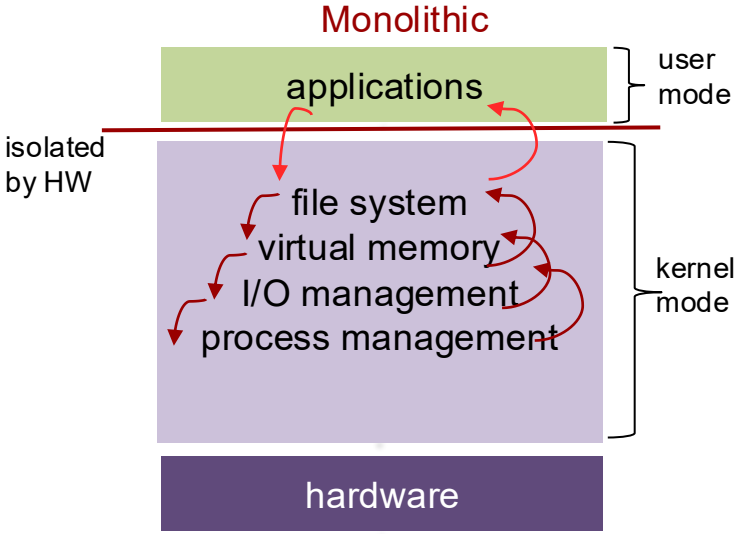


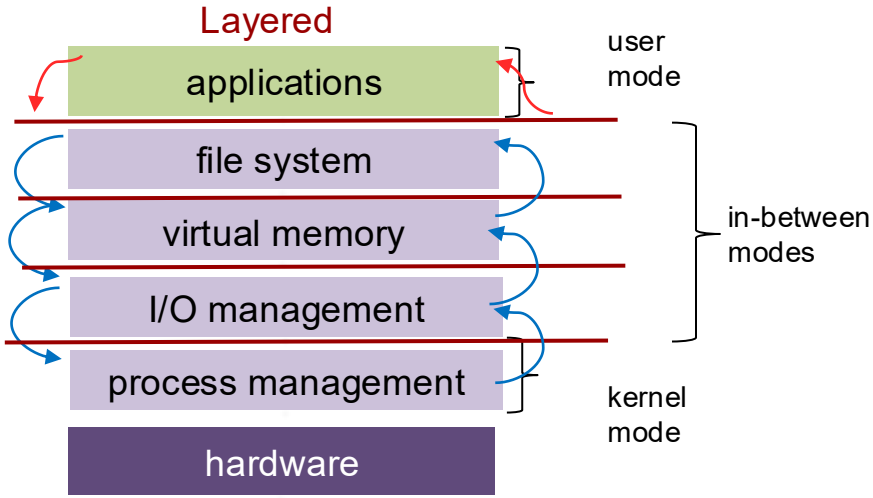
Virtual Machine

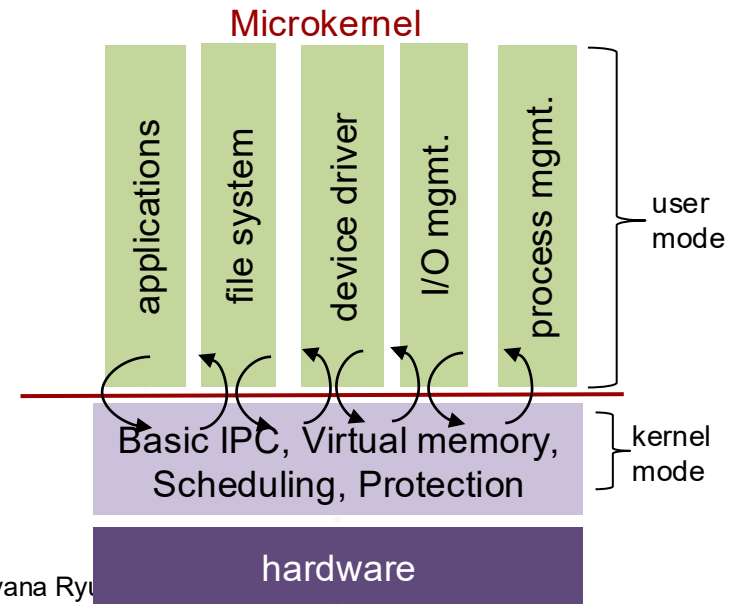
©Tatyana Ryutov

USC Viterbi School of Engineering

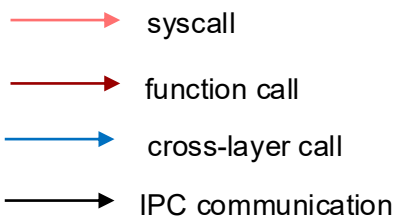
Kernel Organization Approaches and Tradeoffs







	Performance	Reliability Security	Flexibility Comprehensibility
Monolithic	best	worst	worst
Layered	bad	best	good
Microkernel	bad	good	best

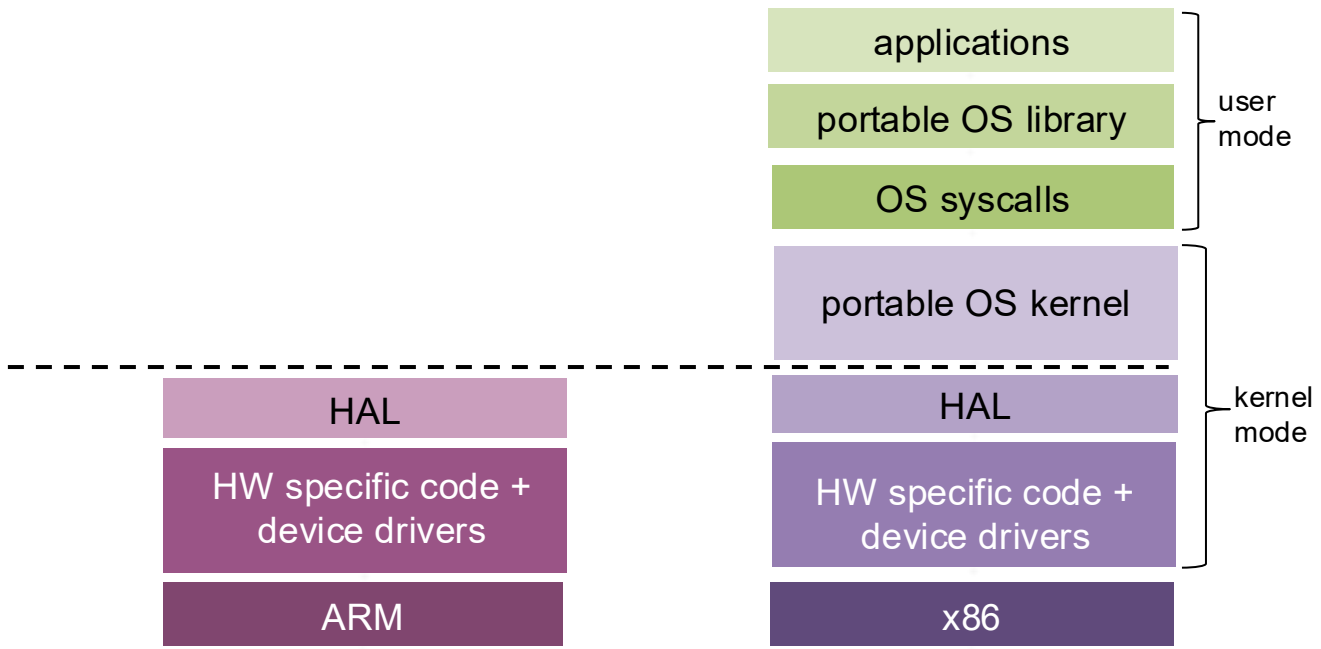


©Tatyana Ryu

USC Viterbi School of Engineering

Monolithic Kernel: Portability

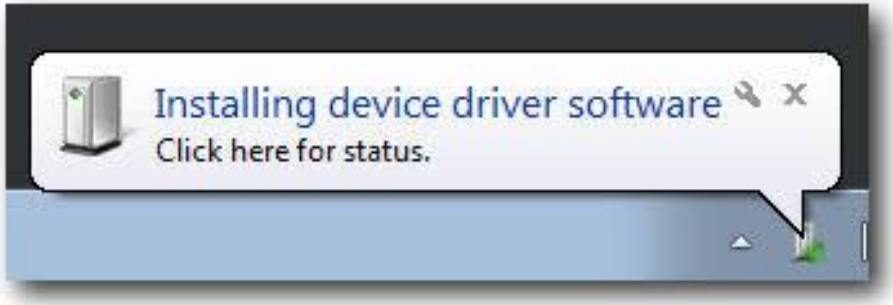
- How do we run OS on different architectures?
- General features to make OS portable and extensible
  1. Hardware Abstraction Layer (HAL)
     - Interface to machine configuration and processor-specific operations
     - Provide functions and abstract models of devices
     - To port OS to a new HW create new implementations of low-level HAL routines
  2. Dynamically loaded device drivers



©Tatyana Ryutov

Monolithic Kernel: Dynamically Installed Device Drivers

- Goal: accommodate a wide variety of physical I/O devices
- Approach:
  - Decouple the OS source code from the specifics of each device
  - Support dynamically loadable device drivers
  - The device manufacturer typically provides the driver code that supports standard interface supported by the kernel
- Tradeoffs?



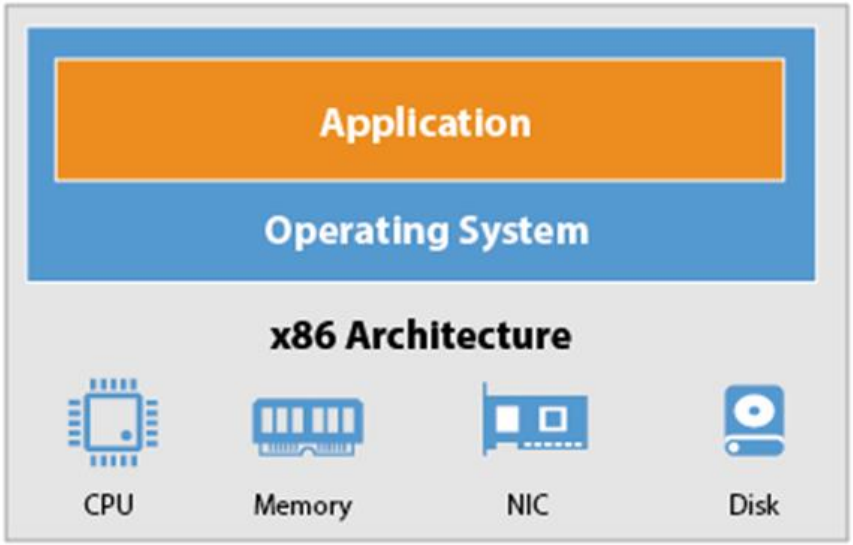
©Tatyana Ryutov

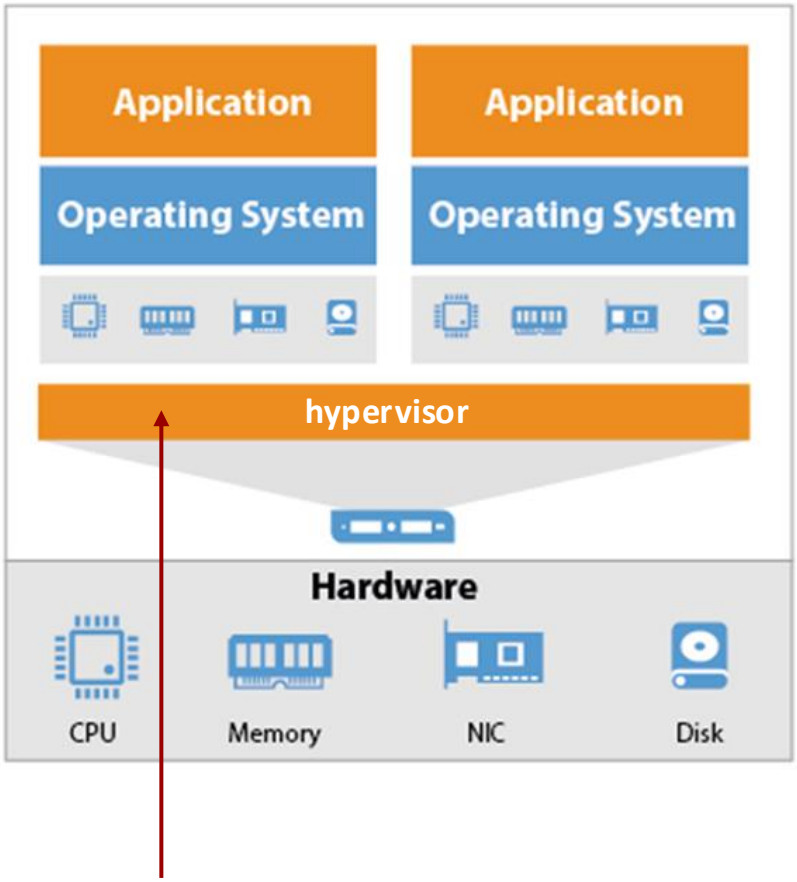
Virtualization Technology

- Virtualization uses software to create an abstraction layer over computer hardware
- Idea: abstract HW of a single computer into different execution environments
  - This creates virtual system (virtual machine, or VM)
- Components:
  - Host – underlying hardware system (or OS)
  - Virtual machine manager (VMM) or hypervisor – manages the underlying HW and provides an abstraction of one or more VMs
  - Guest – software running within a VM
- Single physical machine can run multiple OSs concurrently, each in its own VM

©Tatyana Ryutov

Physical vs. Virtual Machine



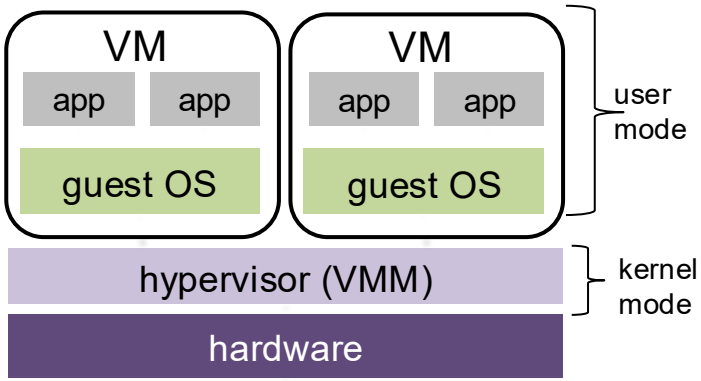


- Extra level of indirection (software abstraction):
  - Enforces isolation
  - Multiplexes physical HW across VMs

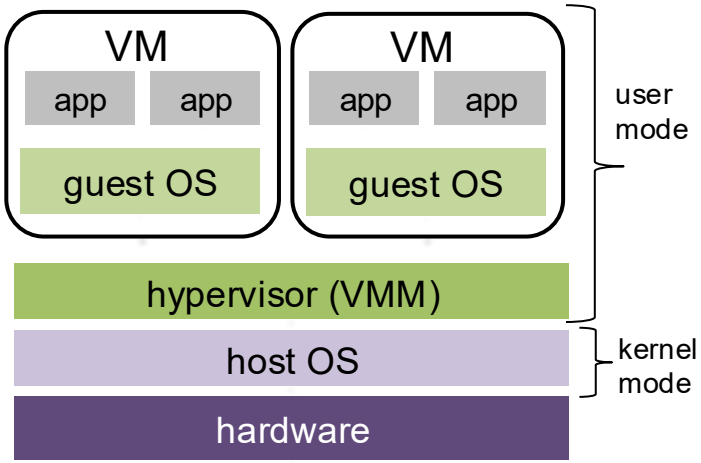
©Tatyana Ryutov

Hypervisor Types

- Native hypervisor (type 1) runs on bare HW
  - E.g., VMware vSphere with ESX/ESXi, Microsoft Hyper-V, Citrix Hypervisor (Xen Server), KVM
- Hosted hypervisor (type 2) runs on top of another OS
  - E.g., VMware Workstation, VMware Player, VirtualBox
- A combination of the native and the hosted approaches
  - Processor virtualization + hosted OS



Type 1



Type 2

©Tatyana Ryutov